MODUL PRAKTIIKUM 01

PERSIAPAN LINGKUNGAN PENGEMBANGAN FLUTTER

Instalasi Dart, Flutter, Android Studio, dan Git



Mata Kuliah	Pemrograman Perangkat Bergerak Lanjut
Modul	I – Persiapan Lingkungan Pengembangan
Tahun	2026

Modul praktikum ini digunakan sebagai panduan kegiatan praktikum mahasiswa.

DAFTAR ISI

A. Tujuan Pembelajaran 3
B. Tugas Pendahuluan 3
C. Dasar Teori 3
Bahasa Pemrograman Dart 3
Framework Flutter 4
Android Studio dan Android SDK 4
Git dan Sistem Kendali Versi 4
Mode Debug, Profil, dan Rilis 4
D. Alat dan Bahan 4
Perangkat Keras (Hardware) 4
Perangkat Lunak (Software) 5
E. Materi Praktikum 5
a. Menjalankan Dart Secara Online (DartPad) 5
b. Menjalankan Dart Secara Offline (Flutter SDK + VS Code) 6
c. Instalasi Flutter SDK pada Sistem Operasi Linux 9
d. Instalasi Flutter SDK pada Sistem Operasi macOS 11
e. Instalasi dan Konfigurasi Git 14
f. Instalasi Android Studio dan Konfigurasi Android SDK 16
g. Memasang Komponen Android SDK 16
h. Memasang Plugin Flutter dan Dart pada Android Studio 18
i. Membuat Perangkat Virtual (Android Emulator) 19
j. Menyiapkan Perangkat Fisik Android (Opsional) 21
1. Mengaktifkan Developer Options 21
2. Mengaktifkan USB Debugging 22
3. Menghubungkan dan Memberikan Izin 22
4. Memeriksa Perangkat pada Terminal 23
5. Khusus pengguna Windows, apabila perangkat tidak terdeteksi, pasang Google USB Driver melalui SDK Manager atau unduh driver resmi dari produsen perangkat. 24
k. Konfigurasi Dukungan Target Web dan Desktop 24
l. Verifikasi Keseluruhan Instalasi dengan flutter doctor 26
m. Instalasi dan Konfigurasi Flutter pada Visual Studio Code 26
n. Sintax umum pada Flutter 31
F. Praktikum 32
a. Praktikum: *Stateless Widget, Stateful Widget,* serta *Layout dan Styling* 32
b. Praktikum Berbantuan AI: Membangun Ulang Aplikasi Widget melalui Prompt Bertahap 42
G. Tugas Praktikum 48
H. Format Pengumpulan Tugas 49

A. Tujuan Pembelajaran

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Menjelaskan peran Dart, Flutter, Android Studio, dan Git dalam pengembangan aplikasi perangkat bergerak.
- Melakukan instalasi Flutter SDK beserta Dart SDK pada sistem operasi Windows, Linux, maupun macOS.
- Melakukan konfigurasi *Environment Variables* (PATH) sehingga perintah dart dan flutter dapat dijalankan melalui terminal.
- Melakukan instalasi dan konfigurasi Git sebagai perangkat lunak kendali versi (*version control*).
- Melakukan instalasi Android Studio, Android SDK, Android SDK *Command-line Tools*, serta membuat *Android Virtual Device* (emulator).
- Melakukan verifikasi kesiapan lingkungan pengembangan menggunakan perintah *flutter doctor* dan memperbaiki masalah yang ditemukan.
- Menjalankan aplikasi Flutter dalam mode debug pada peramban Chrome (web), aplikasi desktop (.exe/Linux/macOS), dan perangkat atau emulator Android.
- Melakukan proses *build* aplikasi Flutter ke seluruh target keluaran (APK, AAB, Web, Windows, Linux, macOS, dan iOS).

B. Tugas Pendahuluan

1. Apa yang dimaksud dengan bahasa pemrograman Dart, dan mengapa Dart digunakan sebagai bahasa utama dalam pengembangan aplikasi Flutter?
2. Jelaskan secara umum apa yang dimaksud dengan Flutter sebagai framework pengembangan aplikasi lintas platform.
3. Apa fungsi Android Studio dan Android SDK dalam proses pengembangan aplikasi Flutter untuk platform Android?
4. Apa yang dimaksud dengan Git dan mengapa Git diperlukan dalam proses pengembangan perangkat lunak, termasuk pada pengembangan aplikasi Flutter?
5. Jelaskan secara singkat perbedaan mendasar antara mode debug, mode profile, dan mode release pada aplikasi Flutter.

C. Dasar Teori

Bahasa Pemrograman Dart

Dart adalah bahasa pemrograman sumber terbuka yang dikembangkan oleh Google dan dirancang khusus untuk pembangunan antarmuka pengguna (UI). Dart bersifat berorientasi objek, memiliki penulisan tipe data yang kuat (*strongly typed*), serta mendukung dua mode kompilasi. Mode *Just-in-Time* (JIT) digunakan pada saat pengembangan sehingga memungkinkan fitur *hot reload*, sedangkan mode *Ahead-of-Time* (AOT) digunakan pada saat rilis sehingga menghasilkan kode mesin asli (*native*) yang berkinerja tinggi. Dart SDK sudah disertakan secara otomatis di dalam Flutter SDK, sehingga mahasiswa tidak perlu melakukan instalasi Dart secara terpisah.

Framework Flutter

Flutter adalah kerangka kerja (*framework*) lintas platform dari Google yang memungkinkan satu basis kode (*single codebase*) dijalankan pada beberapa platform sekaligus, yaitu Android, iOS, Web, Windows, Linux, dan macOS. Flutter tidak menggunakan komponen antarmuka bawaan sistem operasi, melainkan menggambar seluruh antarmukanya sendiri melalui mesin *render* (*rendering engine*) Impeller atau Skia. Seluruh elemen antarmuka pada Flutter disusun sebagai *widget* yang dapat dikomposisikan secara bersarang membentuk widget tree.

Android Studio dan Android SDK

Android Studio merupakan lingkungan pengembangan terpadu (IDE) resmi untuk platform Android. Pada praktikum ini, Android Studio digunakan terutama sebagai penyedia Android SDK, Android SDK *Command-line Tools*, Android SDK Platform-Tools (berisi perintah adb), serta Android Emulator (AVD) yang berfungsi sebagai perangkat virtual untuk menguji aplikasi. Meskipun penulisan kode dapat dilakukan menggunakan Visual Studio Code, komponen Android SDK tetap wajib terpasang agar Flutter dapat melakukan kompilasi dan proses build ke platform Android.

Git dan Sistem Kendali Versi

Git adalah sistem kendali versi terdistribusi (*distributed version control system*) yang berfungsi untuk mencatat setiap perubahan pada berkas proyek, sehingga riwayat pengembangan dapat dilacak dan dikembalikan ke kondisi sebelumnya apabila diperlukan. Selain untuk keperluan kolaborasi melalui layanan seperti GitHub atau GitLab, Git juga merupakan prasyarat teknis bagi Flutter, karena Flutter SDK didistribusikan sebagai repositori Git dan menggunakan Git secara internal pada saat melakukan pembaruan versi (*flutter upgrade*) maupun pengambilan paket dari repositori.

Mode Debug, Profil, dan Rilis

Flutter menyediakan tiga mode kompilasi. Mode debug digunakan pada saat pengembangan, mendukung *hot reload*, namun berukuran besar dan berkinerja lebih lambat. Mode *profile* digunakan untuk mengukur kinerja aplikasi. Mode *release* digunakan untuk distribusi akhir kepada pengguna, dikompilasi secara AOT, berukuran lebih kecil, dan berkinerja optimal. Hasil akhir proses *build* bergantung pada platform tujuan, yaitu berkas APK atau AAB untuk Android, IPA untuk iOS, berkas EXE untuk Windows, bundel aplikasi untuk Linux dan macOS, serta kumpulan berkas HTML, CSS, dan JavaScript untuk Web.

D. Alat dan Bahan

Untuk melaksanakan praktikum pemrograman menggunakan bahasa Dart dan *framework* Flutter, diperlukan perangkat dan perangkat lunak pendukung sebagai berikut:

Perangkat Keras (Hardware)

- Laptop/PC
- Prosesor: Minimal Intel Core i3 atau setara

- RAM: minimal 8 GB untuk Android Studio; 16 GB jika menggunakan Android Emulator
- Penyimpanan: minimal 16 GB ruang kosong untuk Android Studio dan satu emulator; SSD dengan ruang 32 GB atau lebih disarankan
- Layar: Resolusi minimal 1366 × 768 piksel
- Koneksi Internet
- Dibutuhkan untuk instalasi paket, mengakses DartPad, dan mengunduh dependensi Flutter

Perangkat Lunak (Software)

- Flutter SDK pada *stable channel* yang ditetapkan pengampu. Dart SDK sudah termasuk di dalam Flutter SDK sehingga tidak perlu diinstal secara terpisah.
- Git, sebagai prasyarat wajib Flutter. Pada Windows menggunakan Git for Windows, pada Linux menggunakan paket git, dan pada macOS tersedia melalui Xcode Command Line Tools.
- Android Studio, beserta komponen Android SDK Platform, Android SDK *Command-line Tools*, Android SDK Platform-Tools, dan Android Emulator.
- Visual Studio Code, beserta ekstensi Flutter dan ekstensi Dart dari Extensions Marketplace.
- Google Chrome, digunakan sebagai perangkat target untuk menjalankan aplikasi Flutter pada platform Web.
- Khusus Windows: Visual Studio 2022 (*Community Edition*) dengan *workload Desktop development with* C++, diperlukan untuk membangun aplikasi Windows berformat .exe.
- Khusus Linux: paket clang, cmake, ninja-build, pkg-config, libgtk-3-dev, dan libstdc++-12-dev, diperlukan untuk membangun aplikasi desktop Linux.
- Khusus macOS: Xcode beserta CocoaPods, diperlukan untuk membangun aplikasi iOS dan macOS.
- Akun GitHub (opsional), digunakan untuk penyimpanan kode program secara daring.

E. Materi Praktikum

Menjalankan program Dart dapat dilakukan dengan dua metode, yaitu secara online melalui DartPad dan secara offline menggunakan Flutter SDK yang sudah terintegrasi dengan Dart SDK.

a. Menjalankan Dart Secara Online (DartPad)

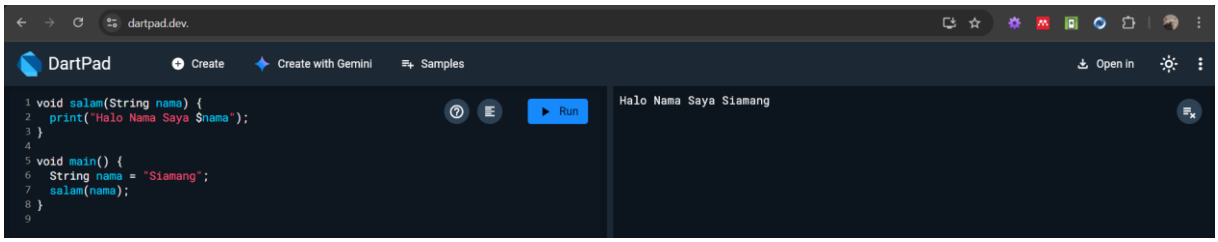
DartPad adalah editor kode berbasis web yang disediakan oleh Google untuk mencoba bahasa Dart tanpa instalasi.

1. Buka peramban (browser) dan akses situs https://dartpad.dev.
2. Pada area editor, ketikkan kode program Dart. Misalnya:

```
void salam(String nama) {
  print("Halo, nama saya $nama");
}

void main() {
  String nama = "Siamang";
  salam(nama);
}
```

3. Klik tombol Run untuk menjalankan program.
4. Hasil output akan ditampilkan pada bagian Console di bawah editor.

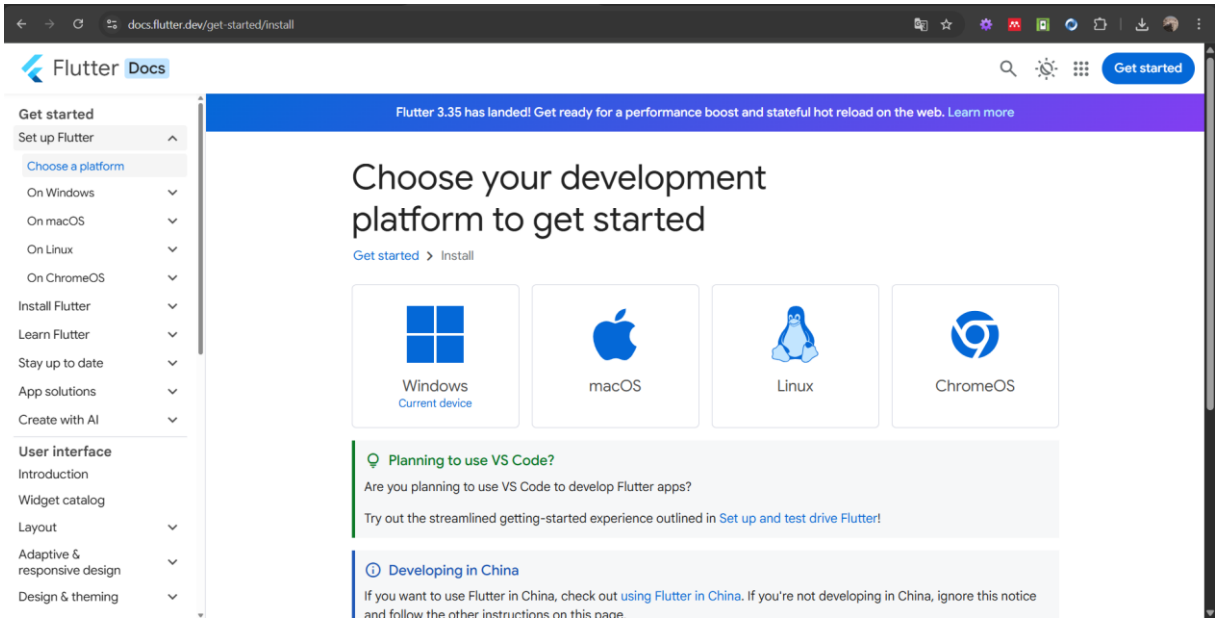


Gambar 1. Menjalankan Dart dengan DartPad

b. Menjalankan Dart Secara Offline (Flutter SDK + VS Code)

Metode ini digunakan jika ingin mengembangkan aplikasi yang lebih kompleks atau menggunakan Flutter.

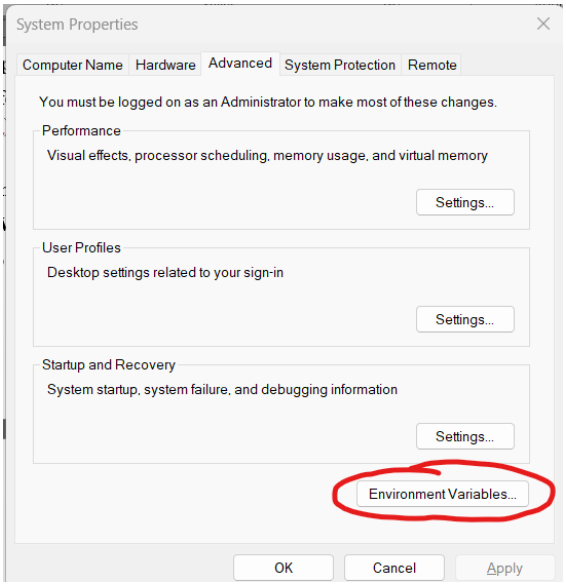
1. Pastikan PC Anda sudah terinstal VS Code, jika belum dapat mengunduh terlebih dahulu pada laman https://code.visualstudio.com
2. Download Flutter SDK dari situs resmi: https://docs.flutter.dev/install/manual
3. Pilih sistem operasi yang Saudara gunakan.



Gambar 2. Pemilihan sistem operasi pada laman unduhan Flutter SDK

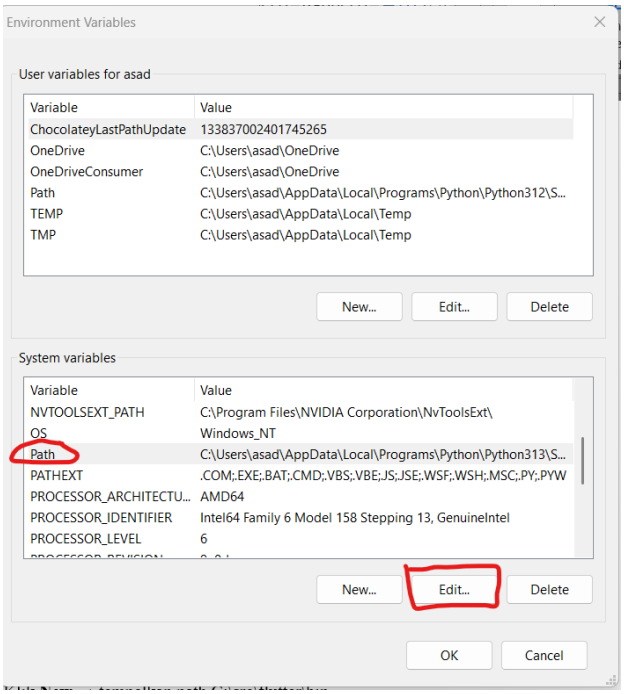
4. Pilih target pengembangan yang dibutuhkan, kemudian unduh Flutter SDK stable untuk sistem operasi Saudara.

5. Ekstrak ke folder, misalnya C: \flutter (Windows) atau /home/user/flutter (Linux/Mac).
6. Tambahkan path flutter/bin ke Environment Variables agar dapat dijalankan melalui terminal. Jika menggunakan Windows, langkahnya sebagai berikut:
7. Tekan Win + R, ketik sysdm.cpl, lalu tekan Enter.
8. Pilih tab Advanced → klik Environment Variables.



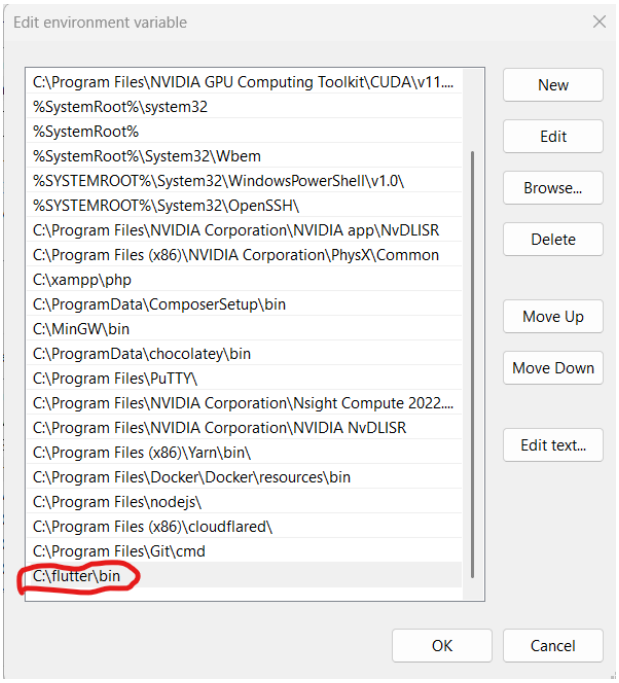
Gambar 3. Tab Advanced pada System Properties dan tombol Environment Variables

9. Pada bagian System variables, cari Path, lalu klik Edit.



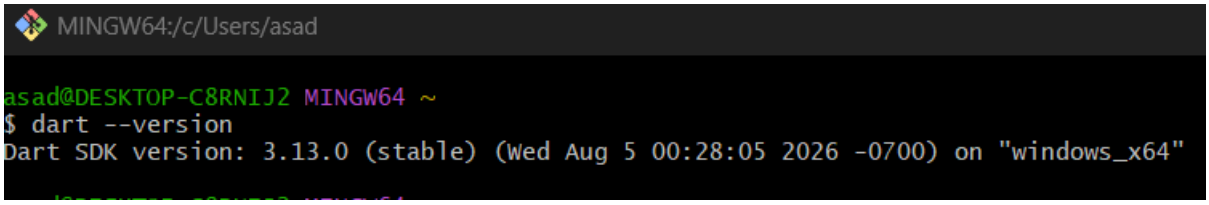
Gambar 4. Memilih variabel Path pada bagian System variables

10. Klik New → tempelkan path C:\flutter\bin.



Gambar 5. Menambahkan path C:\flutter\bin melalui tombol New

11. Klik OK sampai semua jendela tertutup.
12. Untuk mengecek apakah Dart sudah terinstal atau belum, buka terminal kemudian ketik perintah **dart --version**



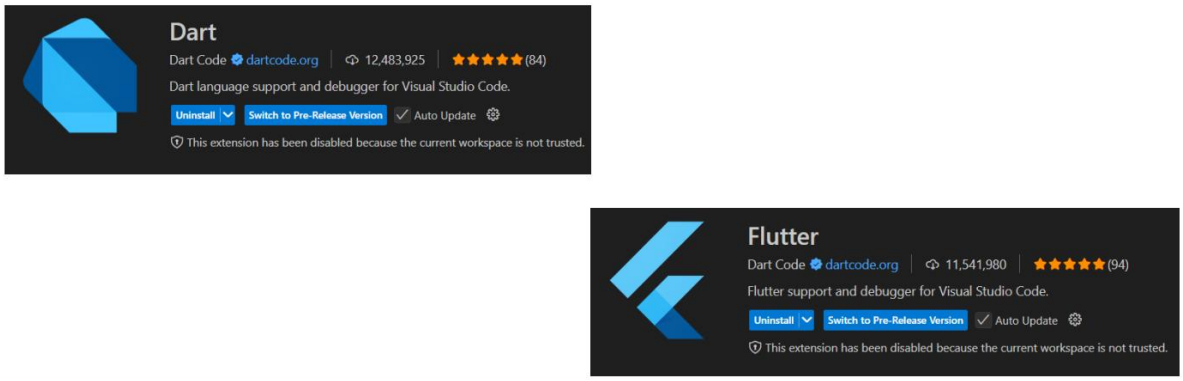
Gambar 6. Pemeriksaan versi Dart melalui perintah dart --version

13. Selain itu, pastikan juga apakah framework Flutter sudah terinstal atau belum dengan membuka terminal, lalu ketik perintah **flutter --version**



Gambar 7. Pemeriksaan versi Flutter melalui perintah flutter --version

14. Tambahkan ekstensi Flutter dan Dart pada Visual Studio Code melalui Extensions Marketplace.



Gambar 8. Pemasangan ekstensi Flutter dan Dart pada Visual Studio Code

c. Instalasi Flutter SDK pada Sistem Operasi Linux

Langkah berikut diujikan pada distribusi berbasis Debian/Ubuntu. Pada distribusi lain, sesuaikan perintah manajer paket yang digunakan (misalnya dnf untuk Fedora atau pacman untuk Arch Linux).

1. Buka Terminal dengan menekan **Ctrl + Alt + T**.
2. Perbarui daftar paket sistem dengan mengetikkan perintah berikut, lalu tekan Enter dan masukkan kata sandi (password) pengguna.

```
sudo apt update && sudo apt upgrade -y
```

3. Instal paket prasyarat yang dibutuhkan oleh Flutter.

```
sudo apt install -y curl git unzip xz-utils zip libglu1-mesa
```

```
root@2e0f1a48e0d7:~# sudo apt install build-essential libgl1 -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
build-essential is already the newest version (12.8ubuntu1.1).
build-essential set to manually installed.
The following additional packages will be installed:
  libdrm-amdgpu1 libdrm-common libdrm-intel1 libdrm-nouveau2 libdrm-radeon1
  libdrm2 libgl1-mesa-dri libglapi-mesa libglvnd0 libglx-mesa0 libglx0
  libllvm12 libpciaccess0 libsensors-config libsensors5 libvulkan1
  libwayland-client0 libx11-6 libx11-data libx11-xcb1 libxau6 libxcb-dri2-0
  libxcb-dri3-0 libxcb-glx0 libxcb-present0 libxcb-randr0 libxcb-shm0
  libxcb-sync1 libxcb-xfixes0 libxcb1 libxdmcp6 libxext6 libxfixes3
  libxshmfence1 libxxf86vm1 mesa-vulkan-drivers
Suggested packages:
  pciutils lm-sensors
The following NEW packages will be installed:
  libdrm-amdgpu1 libdrm-common libdrm-intel1 libdrm-nouveau2 libdrm-radeon1
  libdrm2 libgl1 libgl1-mesa-dri libglapi-mesa libglvnd0 libglx-mesa0 libglx0
  libllvm12 libpciaccess0 libsensors-config libsensors5 libvulkan1
  libwayland-client0 libx11-6 libx11-data libx11-xcb1 libxau6 libxcb-dri2-0
  libxcb-dri3-0 libxcb-glx0 libxcb-present0 libxcb-randr0 libxcb-shm0
  libxcb-sync1 libxcb-xfixes0 libxcb1 libxdmcp6 libxext6 libxfixes3
  libxshmfence1 libxxf86vm1 mesa-vulkan-drivers
0 upgraded, 37 newly installed, 0 to remove and 2 not upgraded.
Need to get 37.1 MB of archives.
After this operation, 528 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 libdrm-common al
l 2.4.107-8ubuntu1~20.04.2 [5396 B]
```

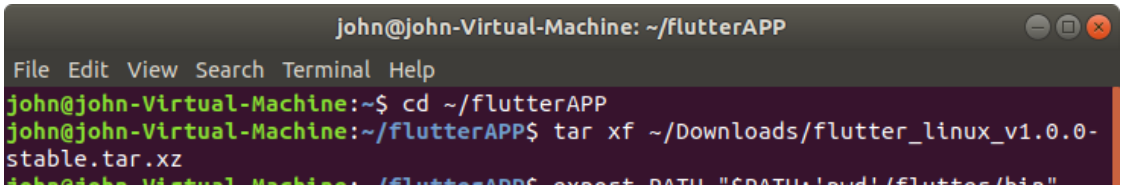
Gambar 9. Proses pemasangan paket menggunakan APT pada Terminal Ubuntu

4. Mengunduh dan Mengekstrak Flutter SDK
5. Buka peramban dan akses https://docs.flutter.dev/get-started/install/linux , kemudian unduh berkas arsip Flutter SDK (berekstensi .tar.xz).
6. Buat direktori kerja untuk menyimpan SDK.

```
mkdir -p ~/development
```

7. Ekstrak berkas hasil unduhan ke direktori tersebut.

```
tar -xf ~/Downloads/flutter_linux_*-stable.tar.xz -C ~/development
```



Gambar 10. Ekstraksi arsip Flutter SDK pada Terminal Ubuntu

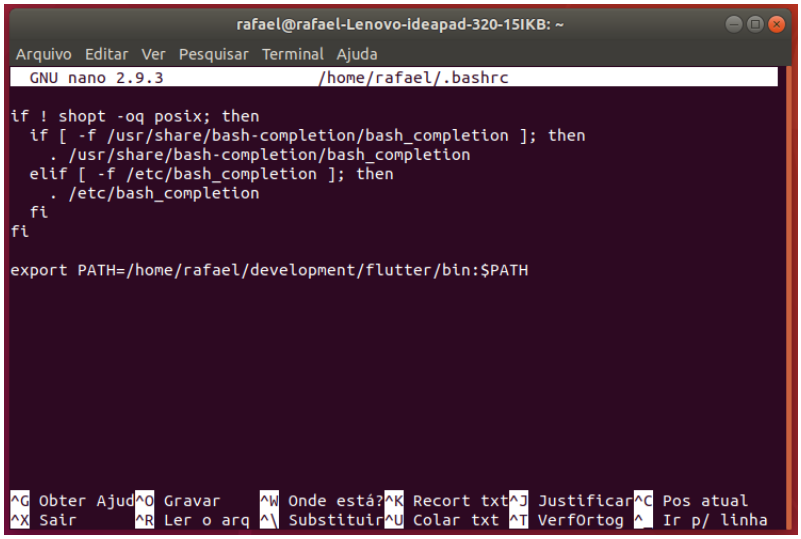
8. Sebagai alternatif, Flutter SDK dapat diambil langsung dari repositori Git resminya dengan perintah: **git clone https://github.com/flutter/flutter.git -b stable ~/development/flutter**

9. Menambahkan Flutter ke Variabel PATH
10. Tambahkan direktori flutter/bin ke dalam berkas konfigurasi shell. Apabila menggunakan Bash, gunakan berkas ~/.bashrc; apabila menggunakan Zsh, gunakan berkas ~/.zshrc.

```
echo 'export PATH="$PATH:$HOME/development/flutter/bin"' >> ~/.bashrc
```

11. Muat ulang konfigurasi shell agar perubahan langsung berlaku.

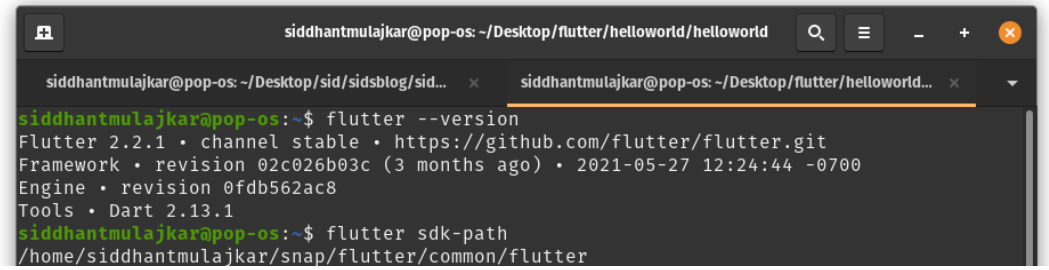
```
source ~/.bashrc
```



Gambar 11. Direktori Flutter SDK ke PATH melalui berkas

12. Verifikasi hasil instalasi dengan menjalankan kedua perintah berikut.

```
flutter --version
dart --version
```



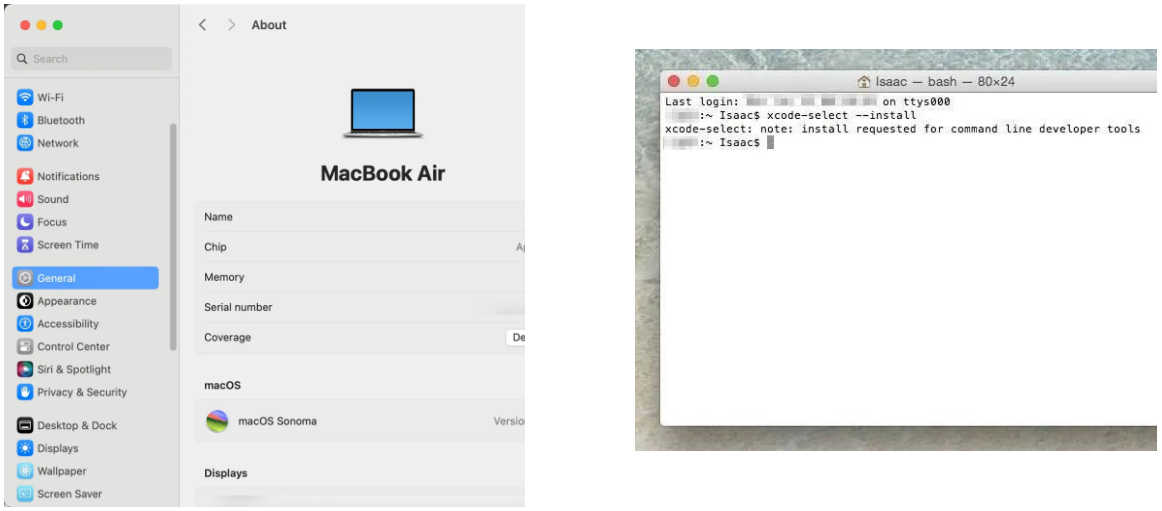
Gambar 12. Verifikasi versi Flutter dan lokasi SDK pada Terminal Linux

CATATAN pada distribusi yang mendukung Snap, instalasi juga dapat dilakukan secara ringkas dengan perintah sudo snap install flutter --classic. Namun cara manual di atas lebih dianjurkan karena memudahkan pengelolaan versi SDK.

d. Instalasi Flutter SDK pada Sistem Operasi macOS

1. Periksa jenis prosesor Mac
2. Klik logo Apple di sudut kiri atas → **About This Mac**. Perhatikan baris Chip atau Processor. Apple M1/M2/M3/M4 termasuk Apple Silicon (arm64); Mac berbasis Intel menggunakan paket x64.
3. Buka Terminal dengan cara **Tekan Command + Spasi**, ketik **Terminal**, lalu tekan Return
4. **Pasang Xcode Command Line Tools,** dengan cara Jalankan perintah berikut. Jendela pemasang akan muncul; pilih Install dan tunggu hingga selesai.

```
xcode-select --install
git --version
```



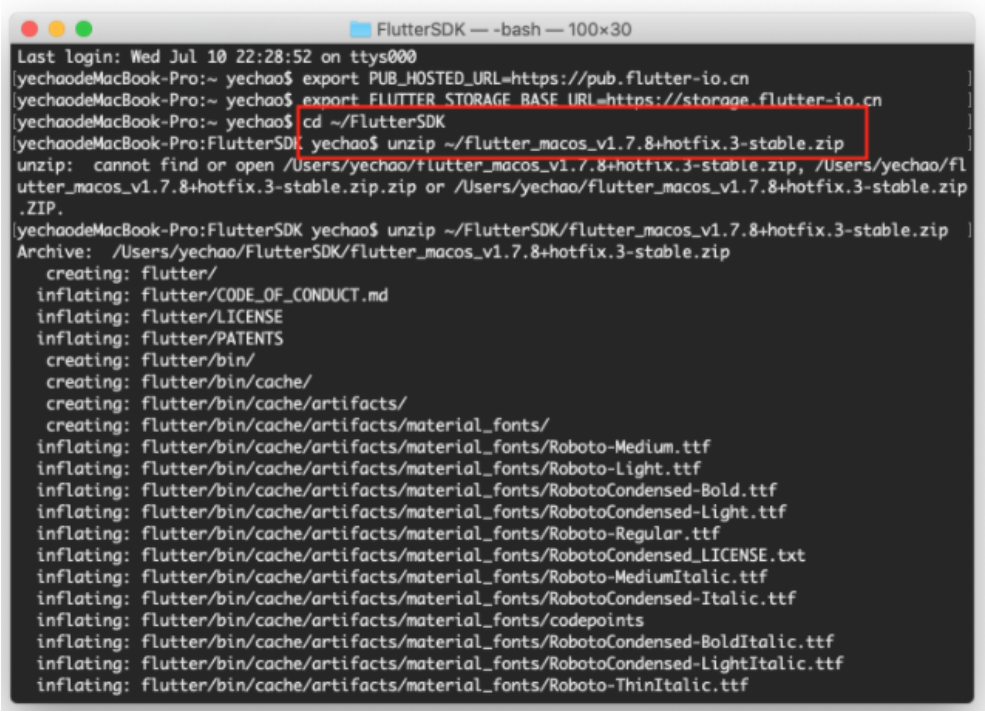
Gambar 13. Pemeriksaan jenis prosesor Mac dan pemasangan Xcode Command Line Tools

5. **Hasil yang diharapkan** Perintah **git --version** menampilkan versi Apple Git. Jika muncul pesan bahwa alat sudah terpasang, langkah ini dapat dilanjutkan.
6. Buka halaman instalasi resmi Flutter, Akses docs.flutter.dev/install/manual, pilih macOS, lalu unduh arsip sesuai prosesor: arm64 untuk Apple Silicon atau x64 untuk Intel.
7. Buka panduan instalasi manual Flutter di https://docs.flutter.dev/install/manual
8. **Buat direktori development,** Direktori ini digunakan untuk menyimpan Flutter SDK agar mudah ditemukan dan dikelola.

```
mkdir -p ~/development
```

9. Ekstrak arsip Flutter, Pastikan nama berkas yang telah diunduh cocok. Tanda * mewakili bagian versi Flutter.

```
unzip ~/Downloads/flutter_macos_*.zip -d ~/development
ls ~/development/flutter
```



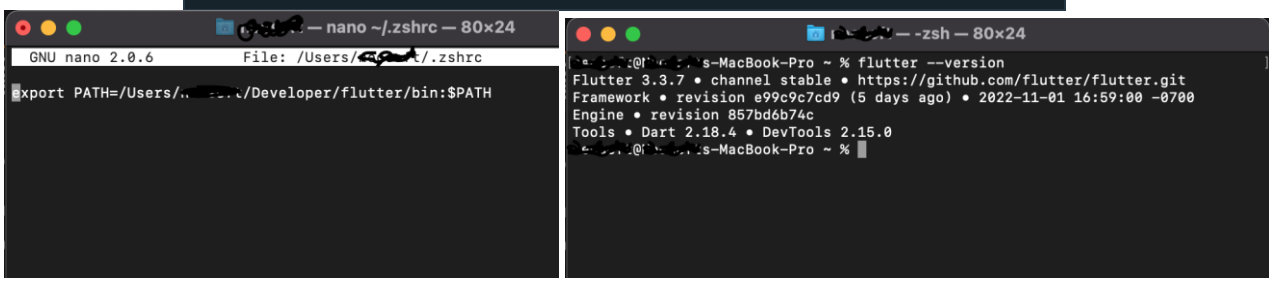
Gambar 14. Proses ekstraksi arsip Flutter SDK pada Terminal macOS

10. **Jika berkas tidak ditemukan** Buka Finder → Downloads dan periksa nama arsipnya. Anda juga dapat mengetik unzip, beri satu spasi, lalu seret berkas ZIP dari Finder ke Terminal.
11. Tambahkan flutter/bin ke konfigurasi Zsh
12. macOS modern menggunakan Zsh secara bawaan. Perintah berikut menambahkan lokasi Flutter secara permanen ke ~/.zshrc.

```
echo 'export PATH="$PATH:$HOME/development/flutter/bin"' >> ~/.zshrc
source ~/.zshrc
```

13. **Pastikan Flutter** dengan cara ketik perintah dan versi Flutter. Hasil versi dapat berbeda dari contoh.

```
which flutter
flutter --version
dart --version
```



Gambar 15. Konfigurasi PATH pada .zshrc dan verifikasi versi Flutter

14. Menyiapkan Xcode untuk Target iOS dan macOS, dengan cara install **Xcode versi lengkap**
15. Unduh Xcode melalui App Store. Sediakan ruang kosong yang cukup karena aplikasi, simulator, dan komponen tambahannya berukuran besar.
16. **Jalankan konfigurasi awal Xcode**
17. Arahkan alat baris perintah ke Xcode, selesaikan komponen awal, dan setujui lisensinya.

```
sudo xcode-select -s /Applications/Xcode.app/Contents/Developer
sudo xcodebuild -runFirstLaunch
sudo xcodebuild -license accept
```

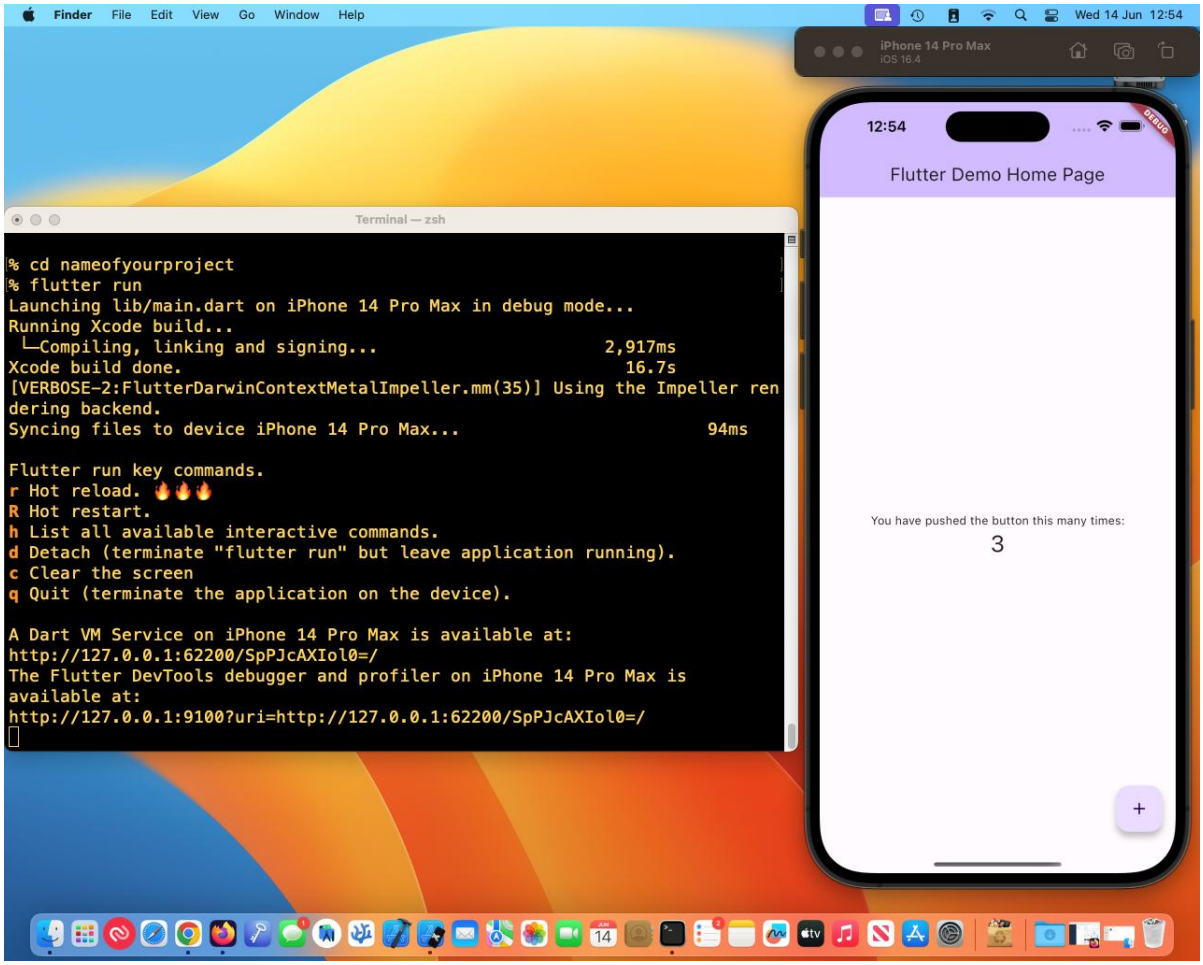
18. **Pasang CocoaPods,** CocoaPods dibutuhkan oleh banyak plugin Flutter lama maupun proyek iOS/macOS yang masih menggunakan Podfile.

```
sudo gem install cocoapods
pod --version
```

CATATAN versi baru Flutter versi baru juga mendukung Swift Package Manager. Namun CocoaPods masih kompatibel dan tetap diperlukan jika proyek atau plugin Anda menggunakannya.

19. Buka Simulator dan periksa perangkat, kemudian pastikan Flutter dapat melihat simulator dan target macOS.

```
open -a Simulator
flutter devices
flutter doctor -v
```



Gambar 16. Aplikasi Flutter berjalan pada iOS Simulator di macOS

e. Instalasi dan Konfigurasi Git

Git diperlukan oleh Flutter untuk mengelola versi SDK dan mengunduh paket dari repositori. Selain itu, Git digunakan mahasiswa untuk menyimpan riwayat pengerjaan kode program praktikum.

Instalasi Git pada Windows

- Buka laman https://git-scm.com/downloads , lalu unduh Git for Windows.
- Jalankan berkas pemasang (installer) hasil unduhan, kemudian klik Next hingga tahap pemilihan komponen.
- Pada tahap Adjusting your PATH environment, pastikan pilihan *Git from the command line and also from 3rd-party software* dalam keadaan terpilih agar Git dapat dijalankan melalui *Command Prompt* dan *PowerShell*.
- Lanjutkan dengan menekan Next hingga muncul tombol Install, lalu tunggu hingga proses instalasi selesai dan klik Finish.

Instalasi Git pada Linux

- Buka Terminal, kemudian jalankan perintah sesuai distribusi yang digunakan.

```
# Debian / Ubuntu
sudo apt install -y git

# Fedora
sudo dnf install -y git

# Arch Linux
sudo pacman -S git
```

Instalasi Git pada macOS

- Git sudah termasuk di dalam *Xcode Command Line Tools*. Apabila belum terpasang, jalankan perintah berikut.

```
sudo xcode-select --install
```

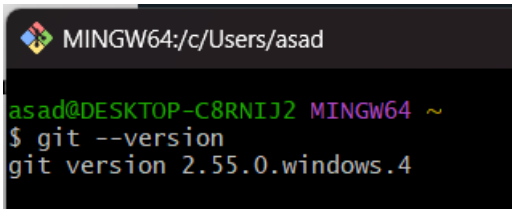
- Alternatif lain adalah memasang melalui Homebrew.

```
brew install git
```

Verifikasi dan Konfigurasi Awal Git

- Periksa versi Git yang terpasang melalui terminal (Command Prompt, PowerShell, atau Terminal).

```
git --version
```



Gambar 17. Version Git

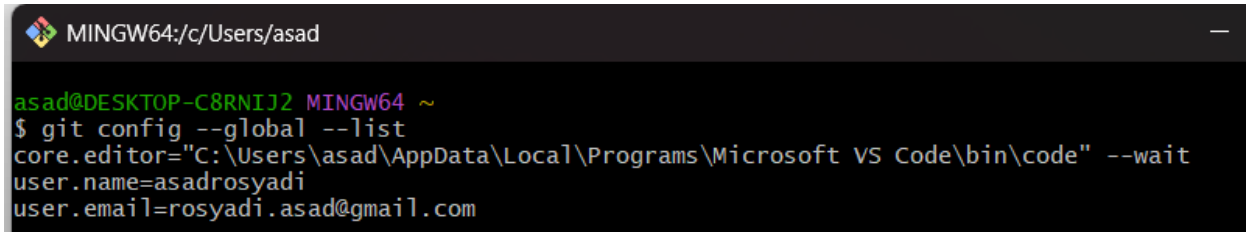
- Atur identitas pengguna yang akan tercatat pada setiap riwayat perubahan (commit). Ganti isi tanda kutip dengan nama dan surel Saudara.

```
git config --global user.name "Nama Mahasiswa"
git config --global user.email "nama@student.ac.id"
```

- Tampilkan kembali konfigurasi untuk memastikan data yang dimasukkan sudah benar.

```
git config --global --list
```

- Jika sudah berhasil tampak sebagaimana Gambar berikut.



Gambar 18. Git Sudah Terhubung ke Akun GitHub

f. Instalasi Android Studio dan Konfigurasi Android SDK

Android Studio digunakan untuk menyediakan Android SDK, Platform-Tools, dan Android Emulator. Ketiga komponen tersebut wajib tersedia agar aplikasi Flutter dapat dijalankan maupun dibangun (*build*) untuk platform Android.

Mengunduh dan Memasang Android Studio

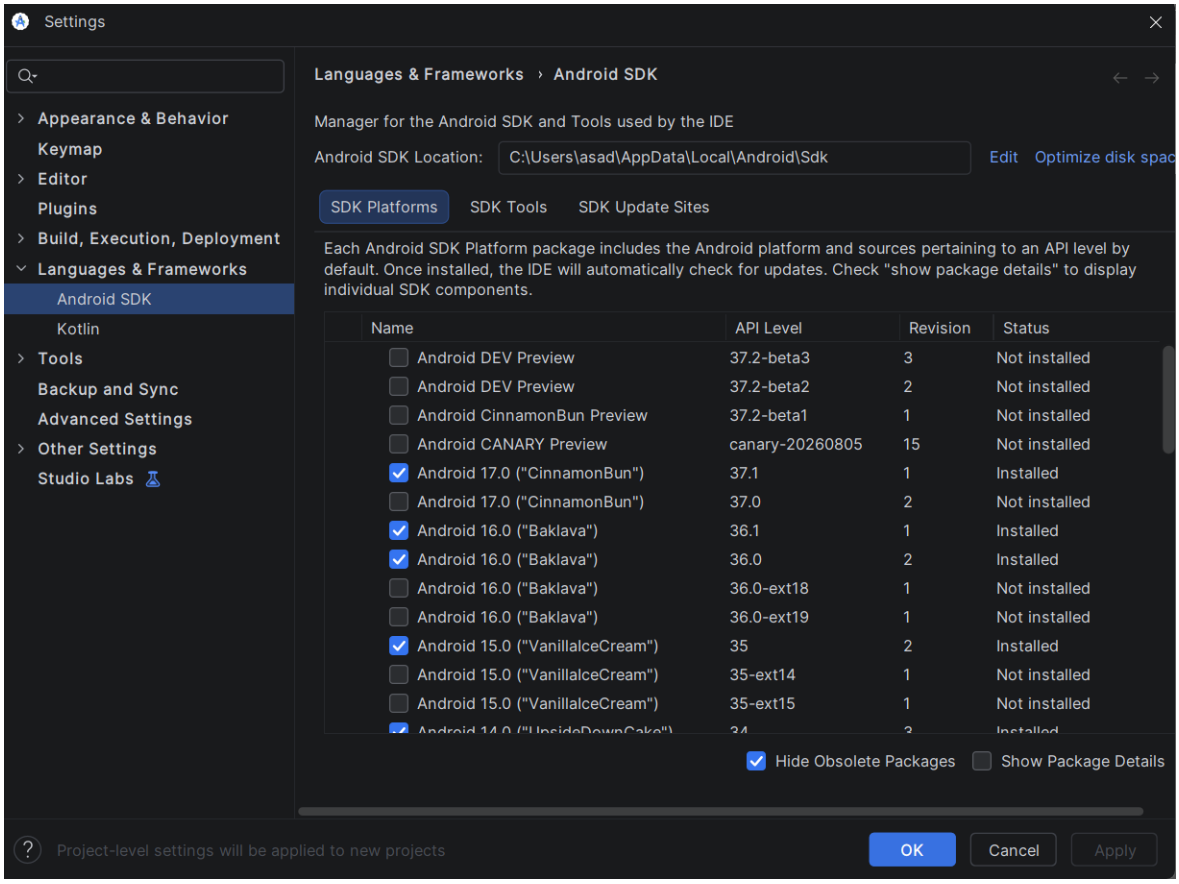
- Buka laman https://developer.android.com/studio , kemudian unduh berkas pemasang sesuai sistem operasi yang digunakan.
- Pada Windows, jalankan berkas .exe hasil unduhan, kemudian tekan Next hingga muncul tombol Install, dan akhiri dengan Finish.
- Pada Linux, ekstrak berkas .tar.gz ke direktori /opt kemudian jalankan skrip studio.sh.

```
sudo tar -xzf ~/Downloads/android-studio-*-linux.tar.gz -C /opt
cd /opt/android-studio/bin
./studio.sh
```

- Alternatif yang lebih singkat pada Linux adalah menggunakan Snap: **sudo snap install android-studio --classic**
- Pada macOS, buka berkas .dmg hasil unduhan, kemudian seret ikon Android Studio ke folder Applications.
- Jalankan Android Studio untuk pertama kali. Pada jendela Setup Wizard, pilih jenis instalasi Standard, setujui seluruh lisensi, lalu tunggu hingga proses pengunduhan komponen selesai.

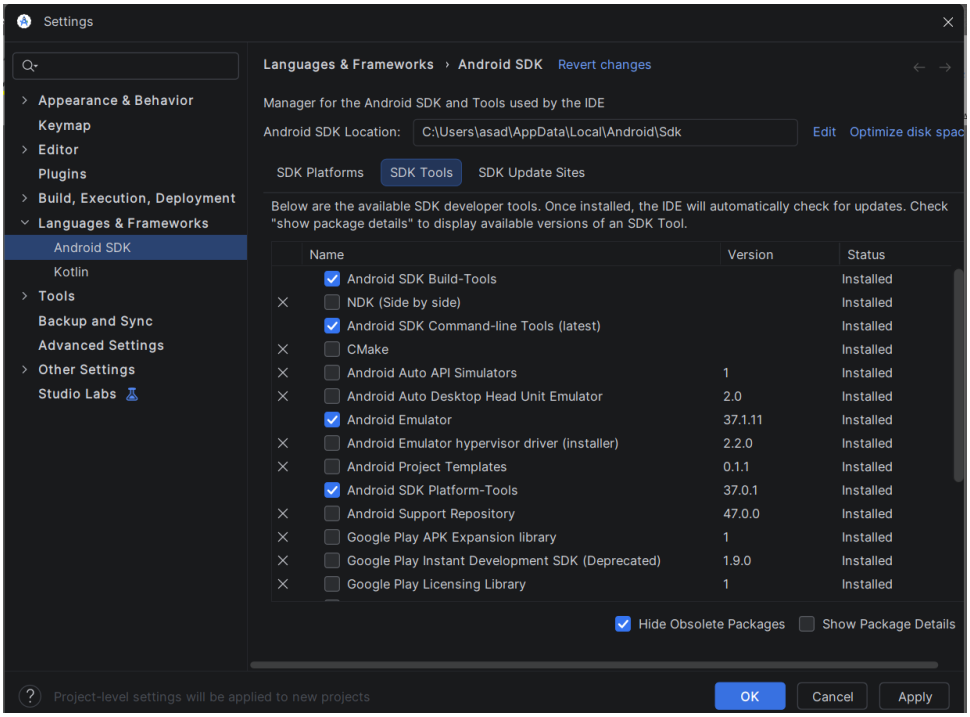
g. Memasang Komponen Android SDK

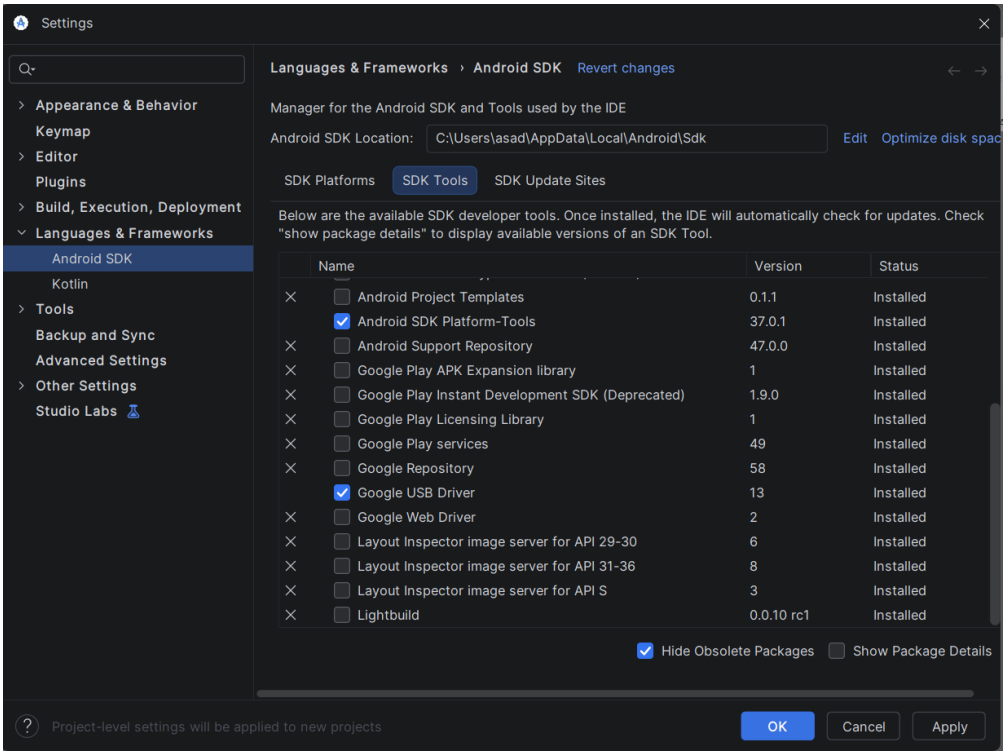
- Pada jendela awal Android Studio, klik More Actions, lalu pilih SDK Manager. Apabila sebuah proyek sudah terbuka, menu tersebut dapat diakses melalui Settings → Languages & Frameworks → Android SDK.
- Pada tab SDK Platforms, centang versi Android stabil terbaru yang tersedia dan dibutuhkan untuk praktikum, pastikan pilih versi yang *stable*.



Gambar 19. Tampilan SDK Platform

- Beralih ke tab SDK Tools, kemudian centang komponen Android SDK Command-line Tools (latest), Android SDK Build-Tools, Android SDK Platform-Tools, dan Android Emulator. Khusus pengguna Windows, centang pula Google USB Driver.

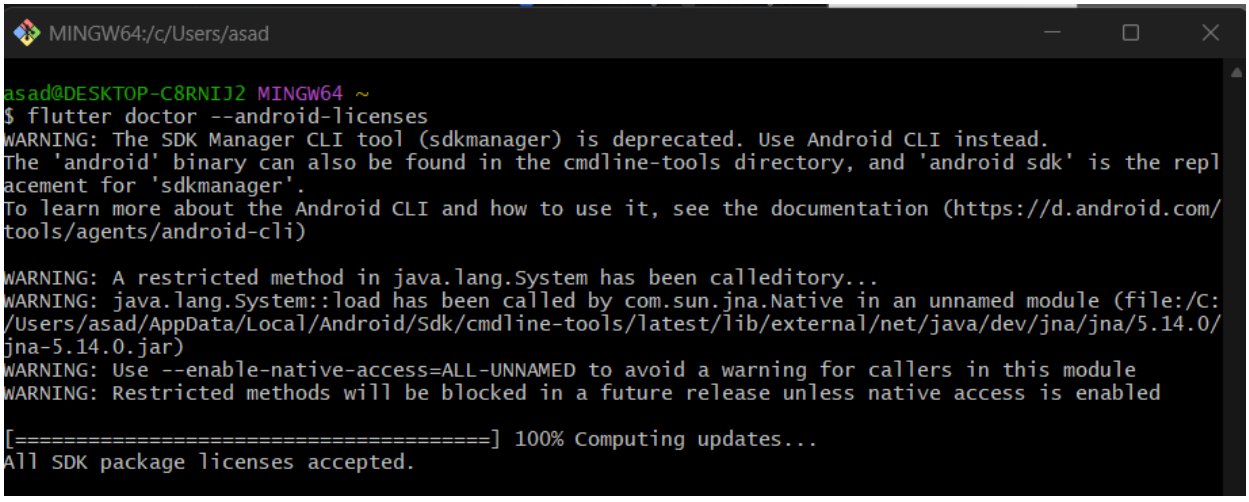




Gambar 20. Tampilan Android SDK Manager pada tab SDK Tools

- Klik Apply, lalu OK, dan tunggu hingga seluruh komponen selesai diunduh.
- Setujui lisensi Android SDK melalui terminal dengan perintah berikut, kemudian ketik y setiap kali muncul konfirmasi.

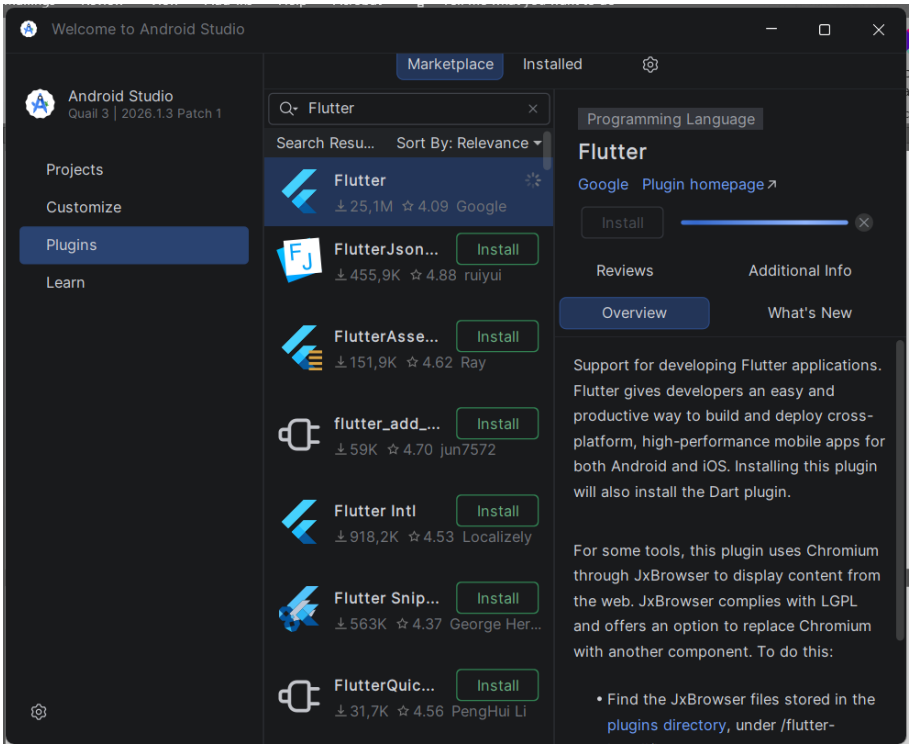
```
flutter doctor --android-licenses
```



Gambar 21. Menyetujui Lisensi Android SDK

h. Memasang Plugin Flutter dan Dart pada Android Studio

- Pada jendela awal Android Studio, pilih More Actions → Plugins. Apabila sudah berada di dalam proyek, pilih menu File → Settings → Plugins (pada macOS: Android Studio → Settings → Plugins).
- Pilih tab Marketplace, ketikkan kata kunci Flutter pada kotak pencarian, lalu klik Install.

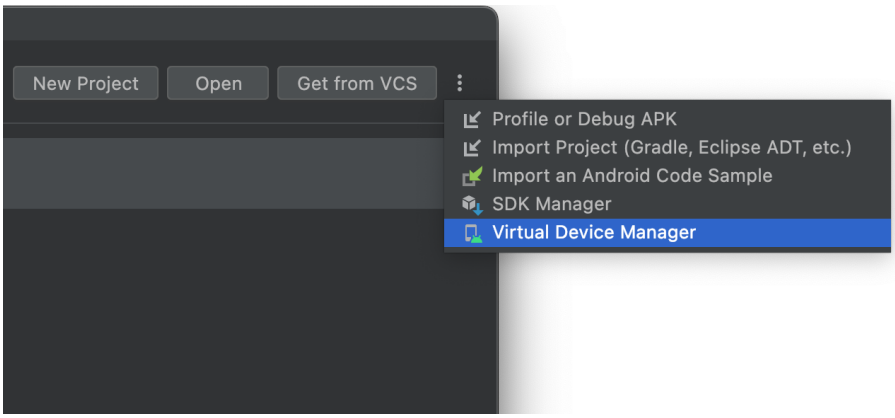


Gambar 22. Install Flutter pada Android Studio

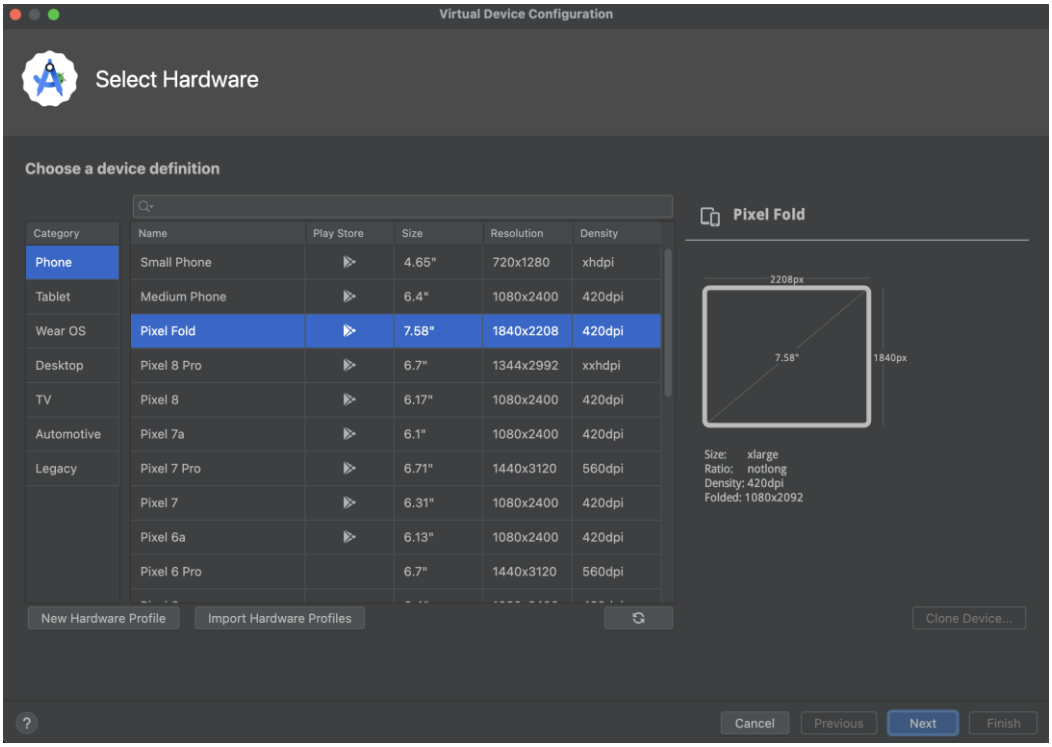
- Plugin Dart akan dipasang secara otomatis sebagai dependensi. Klik Accept apabila muncul konfirmasi.
- Klik Restart IDE agar plugin aktif.

i. Membuat Perangkat Virtual (Android Emulator)

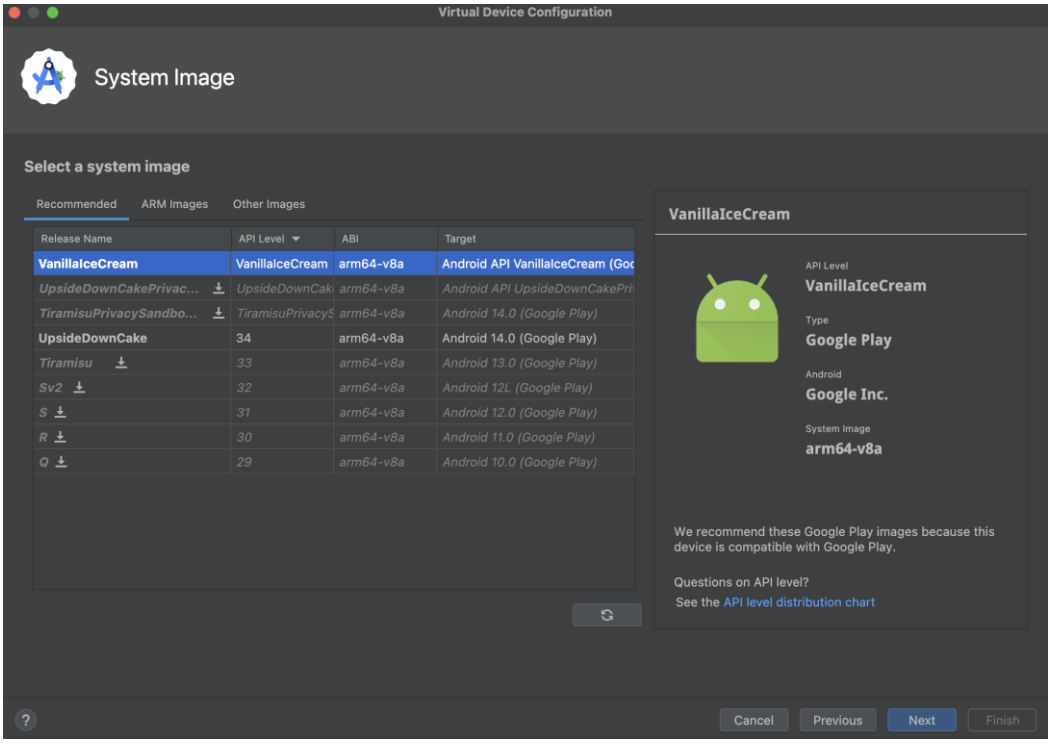
- Pada jendela awal Android Studio, pilih More Actions → Virtual Device Manager. Apabila sudah berada di dalam proyek, pilih menu Tools → Device Manager.
- Klik tombol Create Device (ikon +), kemudian pilih salah satu definisi perangkat, misalnya Pixel 7, lalu klik Next.



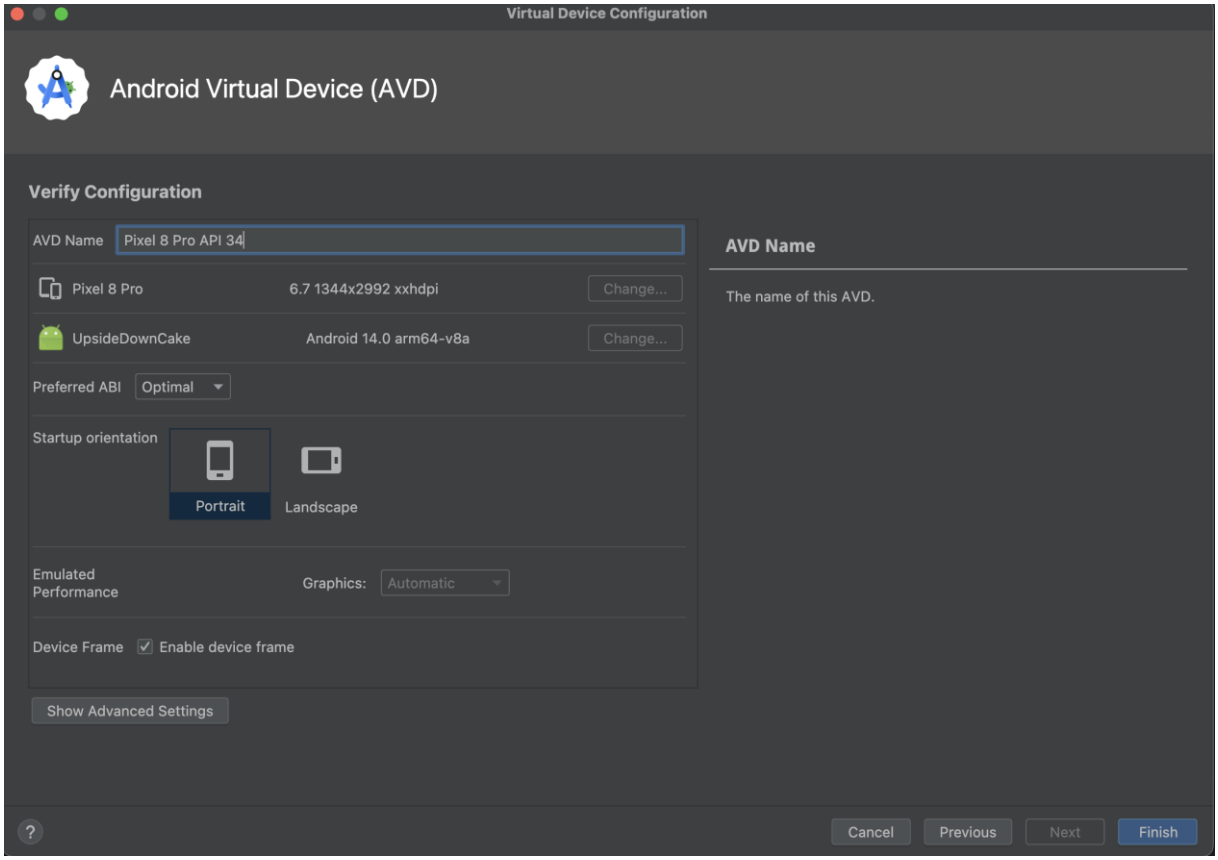
Gambar 23. Membuka Virtual Device Manager dari Welcome Screen Android Studio



Gambar 24. Tampilan asli Select Hardware pada pembuatan Android Virtual Device



Gambar 25. Tampilan asli pemilihan System Image untuk Android Virtual Device



Gambar 26. Tampilan asli verifikasi konfigurasi Android Virtual Device

- Pilih system image yang akan digunakan, misalnya API 35 (Android 15). Klik ikon unduh di samping nama image apabila belum tersedia di komputer, kemudian klik Next.
- Beri nama perangkat virtual pada kolom AVD Name, lalu klik Finish. Perangkat virtual akan muncul pada daftar Device Manager.
- Klik ikon Run (▶) pada daftar perangkat untuk menjalankan emulator. Tunggu hingga layar utama Android tampil sepenuhnya.

CATATAN emulator membutuhkan RAM minimal 16 GB, sehingga jika RAM tidak sampai 16GB, maka tidak perlu install

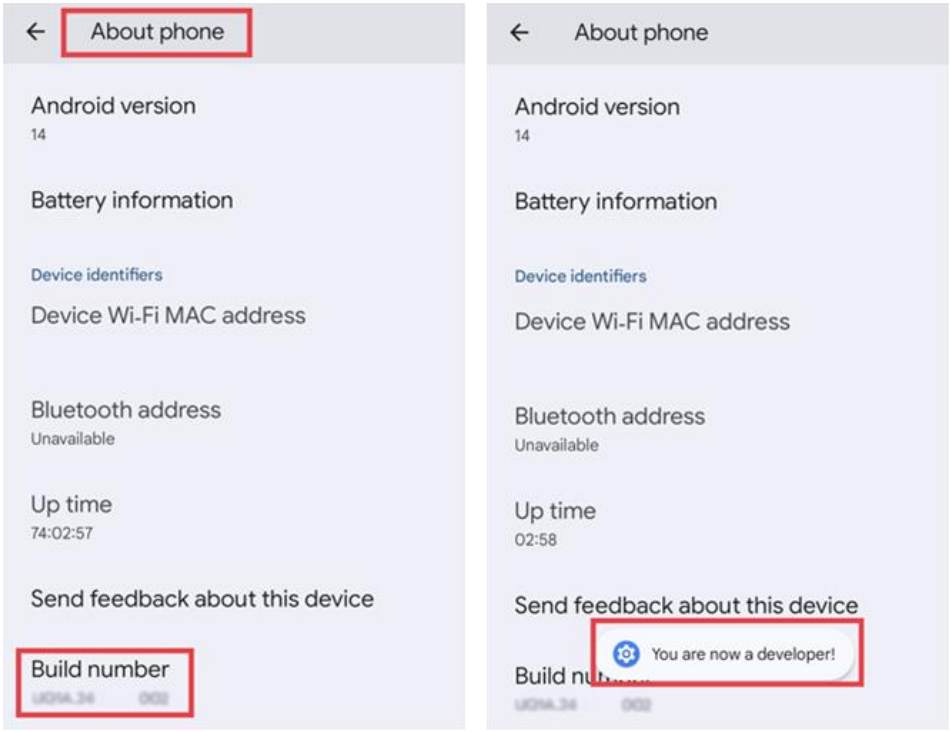
- Untuk Linux, tambahkan sintax berikut untuk menyiapkan KVM (*Kernel-based Virtual Machine*) agar Android Emulator dapat berjalan lebih cepat pada Linux.

```
sudo apt install -y qemu-kvm
sudo adduser $USER kvm
```

j. Menyiapkan Perangkat Fisik Android (Opsional)

1. Mengaktifkan Developer Options
    - Android umum/Google Pixel: Buka Settings → About phone → Build number.
    - Samsung: Buka Settings → About phone → Software information → Build number.

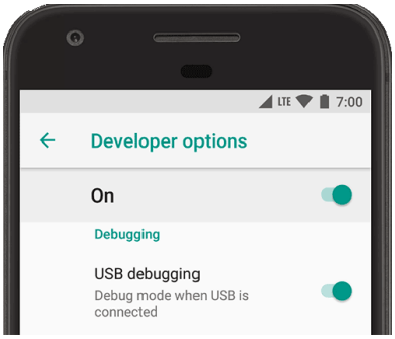
- Tindakan: Ketuk Build number sebanyak tujuh kali. Masukkan PIN atau pola layar jika diminta. Pesan “You are now a developer!” menandakan mode pengembang telah aktif.



Gambar 27. Lokasi Build number dan pesan bahwa Developer Options telah aktif

2. **Mengaktifkan USB Debugging**

- Buka menu: Kembali ke Settings, lalu buka System → Developer options. Jika menu sulit ditemukan, gunakan kolom pencarian Settings dan ketik “Developer options”.
- **Aktifkan:** Pastikan sakelar utama Developer options berada pada posisi On, kemudian aktifkan USB debugging pada bagian Debugging.
- **Variasi tampilan** Pada beberapa perangkat Samsung, Xiaomi, OPPO, vivo, atau realme, Developer options dapat berada di menu Additional settings, System settings, atau pengaturan tambahan lainnya.

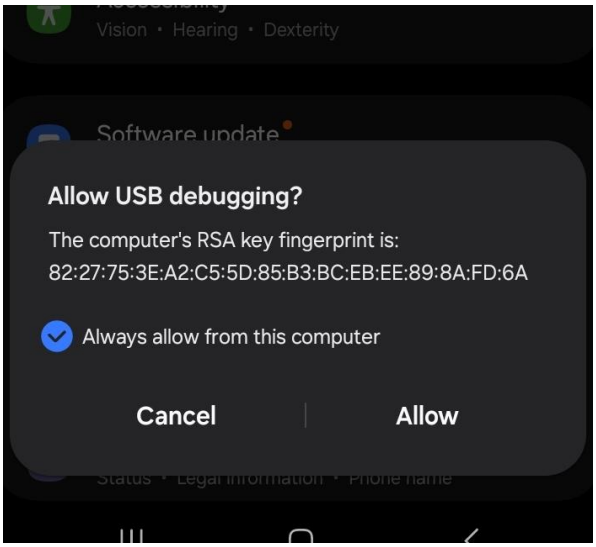


Gambar 28. Sakelar USB debugging pada menu Developer options

3. **Menghubungkan dan Memberikan Izin**

- **Hubungkan**: Sambungkan smartphone ke komputer menggunakan kabel USB data, kemudian buka kunci layar smartphone.

- **Mode USB**: Jika perangkat belum terbaca, buka notifikasi USB dan pilih File transfer/Android Auto atau Transferring files, bukan Charge only.
- **Berikan izin:** Pada dialog Allow USB debugging?, periksa nama komputer atau sidik jari RSA, centang Always allow from this computer hanya untuk komputer pribadi, lalu pilih Allow.



Gambar 29. Dialog konfirmasi RSA untuk mengizinkan USB debugging

- Jika dialog tidak muncul Cabut dan pasang kembali kabel, ubah mode USB menjadi File transfer, lalu jalankan adb devices. Pastikan layar smartphone tetap terbuka.

4. **Memeriksa Perangkat pada Terminal**

- **Lokasi menjalankan**: Buka Terminal/Command Prompt/PowerShell pada komputer. Perintah dapat dijalankan dari folder mana pun apabila adb dan Flutter sudah masuk ke PATH.
- **Jalankan:** Gunakan kedua perintah berikut secara berurutan.

```
adb devices
flutter devices
```

- **Hasil adb:** Nomor seri perangkat harus muncul dengan status device.
- **Hasil Flutter:** Nama perangkat harus muncul sebagai mobile dengan platform android-arm atau android-arm64.





Gambar 30. Contoh perangkat Android yang berhasil terdeteksi oleh ADB dan Flutter

5. Khusus pengguna Windows, apabila perangkat tidak terdeteksi, pasang Google USB Driver melalui SDK Manager atau unduh driver resmi dari produsen perangkat.

k. Konfigurasi Dukungan Target Web dan Desktop

Flutter mampu mengompilasi satu basis kode (*single code base*) menjadi aplikasi Android, iOS, web, Windows, Linux, dan macOS. Setiap target memerlukan perkakas (*toolchain*) tambahan yang berbeda, sehingga perlu disiapkan terlebih dahulu sebelum proses debug maupun build dijalankan.

1. Target Web (Google Chrome)

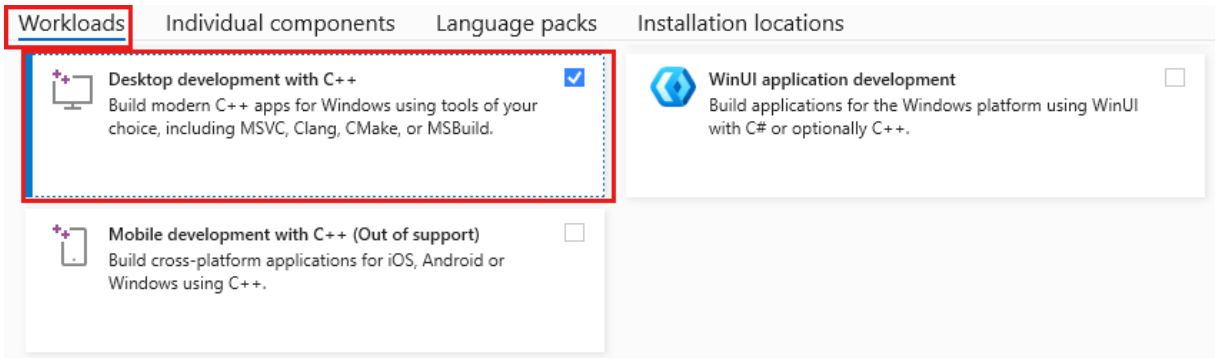
- Pastikan peramban Google Chrome sudah terpasang. Apabila belum, unduh melalui https://www.google.com/chrome .
- Pada instalasi Flutter terbaru, dukungan web umumnya sudah aktif. Jika Chrome belum muncul pada hasil flutter devices, aktifkan dukungan web melalui terminal.

```
flutter config --enable-web
```

- Pastikan Chrome telah terdaftar sebagai perangkat sasaran dengan menjalankan perintah flutter devices.

2. **Target Windows (Berkas .exe)**

- Unduh Visual Studio 2022 Community melalui https://visualstudio.microsoft.com/downloads . Perlu diperhatikan bahwa Visual Studio berbeda dengan Visual Studio Code.
- Jalankan Visual Studio Installer, kemudian pada tab Workloads centang Desktop development with C++.
- Pastikan komponen MSVC C++ build tools terbaru, Windows SDK, dan C++ CMake tools for Windows tercentang pada panel Installation details.



Gambar 31. Visual Studio Installer dengan workload Desktop development with C++ terpilih

- Klik Install, tunggu proses selesai, kemudian mulai ulang (restart) komputer.
- Aktifkan dukungan desktop Windows pada Flutter.

```
flutter config --enable-windows-desktop
```

3. **Target Linux**

- Pasang pustaka pengembangan yang dibutuhkan untuk membangun aplikasi desktop Linux.

```
sudo apt install -y clang cmake ninja-build pkg-config libgtk-3-dev liblzma-dev libstdc++-12-dev
```

- Aktifkan dukungan desktop Linux pada Flutter.

```
flutter config --enable-linux-desktop
```

4. **Target macOS dan iOS**

- Pastikan Xcode beserta CocoaPods sudah terpasang sebagaimana dijelaskan pada langkah instalasi macOS.
- Aktifkan dukungan desktop macOS pada Flutter.

```
flutter config --enable-macos-desktop
```

- Jalankan iOS Simulator apabila akan melakukan pengujian pada perangkat iOS virtual.

```
open -a Simulator
```

l. Verifikasi Keseluruhan Instalasi dengan flutter doctor

Perintah flutter doctor berfungsi memeriksa kelengkapan seluruh komponen yang telah dipasang serta memberikan petunjuk perbaikan apabila masih terdapat komponen yang belum lengkap.

Jalankan perintah berikut pada terminal.

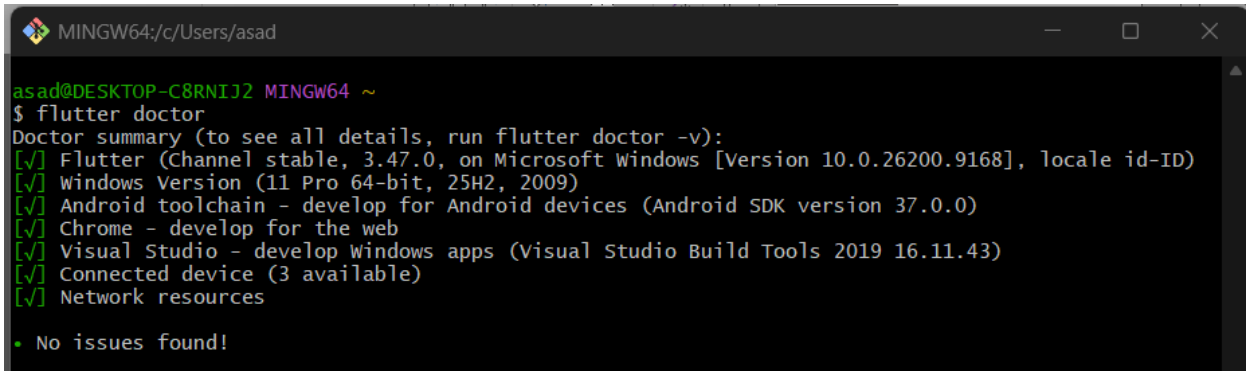
```
flutter doctor
```

Perhatikan tanda yang muncul pada setiap baris hasil pemeriksaan.

Tanda [✓] berarti komponen tersebut sudah terpasang dengan benar.

Tanda [!] berarti komponen sudah terpasang, tetapi masih terdapat peringatan atau konfigurasi opsional yang belum lengkap.

Tanda [✗] berarti komponen belum terpasang dan wajib diperbaiki sebelum melanjutkan praktikum.



Gambar 32. Hasil Flutter Doctor tanpa error

Perbaiki setiap komponen yang bertanda [✗] atau [!] sesuai petunjuk yang ditampilkan, kemudian jalankan kembali flutter doctor hingga seluruh komponen yang dibutuhkan bertanda [✓].

> **CATATAN** tanda [!] pada baris Android Studio maupun Xcode dapat diabaikan apabila target platform yang bersangkutan memang tidak digunakan. Namun baris Flutter dan Android toolchain wajib bertanda [✓].

m. Instalasi dan Konfigurasi Flutter pada Visual Studio Code

Visual Studio Code (VS Code) merupakan editor kode ringan yang paling banyak digunakan untuk pengembangan aplikasi Flutter. Berbeda dengan Android Studio, VS Code tidak menyediakan Android SDK, sehingga Flutter SDK dan Android SDK tetap harus dipasang terlebih dahulu sebagaimana dijelaskan pada langkah sebelumnya. Bagian ini menjelaskan proses pemasangan VS Code beserta ekstensi Flutter dan Dart, penentuan lokasi Flutter SDK, pembuatan proyek, hingga menjalankan aplikasi pertama.

1. Mengunduh dan Memasang Visual Studio Code

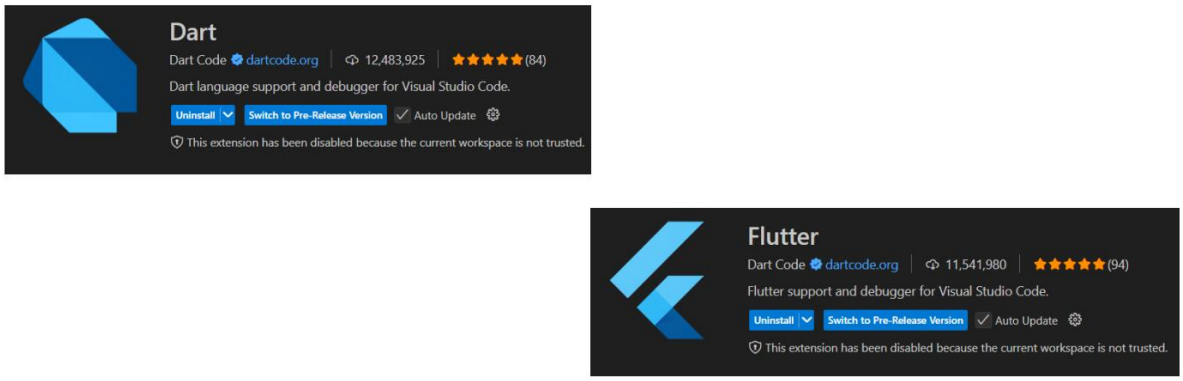
- Buka laman https://code.visualstudio.com , kemudian unduh berkas pemasang sesuai sistem operasi yang digunakan (Windows, Linux, atau macOS).
- Pada Windows, jalankan berkas .exe hasil unduhan, centang pilihan Add to PATH pada tahap Select Additional Tasks, lalu tekan Next hingga muncul tombol Install.
- Pada Linux, pasang berkas .deb dengan perintah berikut, atau gunakan Snap dengan perintah **sudo snap install code --classic**.

```
sudo apt install -y ./code_*_amd64.deb
```

- Pada macOS, ekstrak berkas .zip hasil unduhan, kemudian seret aplikasi Visual Studio Code ke folder Applications.

2. Memasang Ekstensi Flutter dan Dart

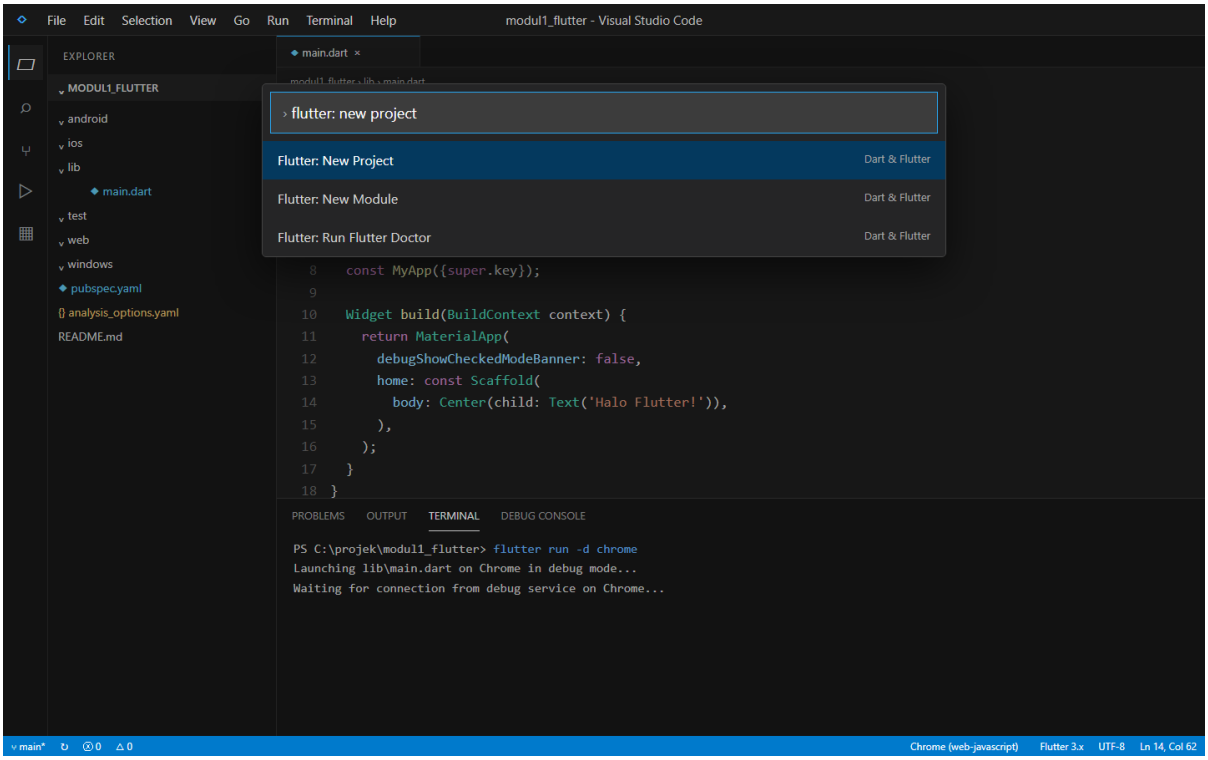
- Jalankan Visual Studio Code, kemudian buka panel Extensions dengan menekan Ctrl + Shift + X (pada macOS: Command + Shift + X) atau melalui ikon Extensions pada Activity Bar di sisi kiri.
- Ketikkan kata kunci Flutter pada kotak pencarian, pilih ekstensi Flutter yang diterbitkan oleh Dart Code, lalu klik Install.
- Ekstensi Dart akan terpasang secara otomatis karena merupakan dependensi dari ekstensi Flutter, sehingga tidak perlu dipasang secara terpisah.
- Tunggu hingga status tombol berubah menjadi Uninstall atau muncul ikon roda gigi, yang menandakan ekstensi telah terpasang.



Gambar 33. Pemasangan ekstensi Flutter dan Dart melalui Extensions Marketplace

3. Membuat Proyek Flutter Baru

- Buka Command Palette, ketikkan Flutter: New Project, lalu tekan Enter.
- Pilih jenis proyek Application, kemudian tentukan folder penyimpanan proyek melalui jendela penjelajah berkas.
- Ketikkan nama proyek menggunakan huruf kecil dan garis bawah, misalnya modul1_flutter, lalu tekan Enter. Nama proyek tidak boleh mengandung spasi maupun huruf kapital.
- Tunggu hingga proses pembuatan proyek selesai. Berkas utama proyek berada pada lib/main.dart.



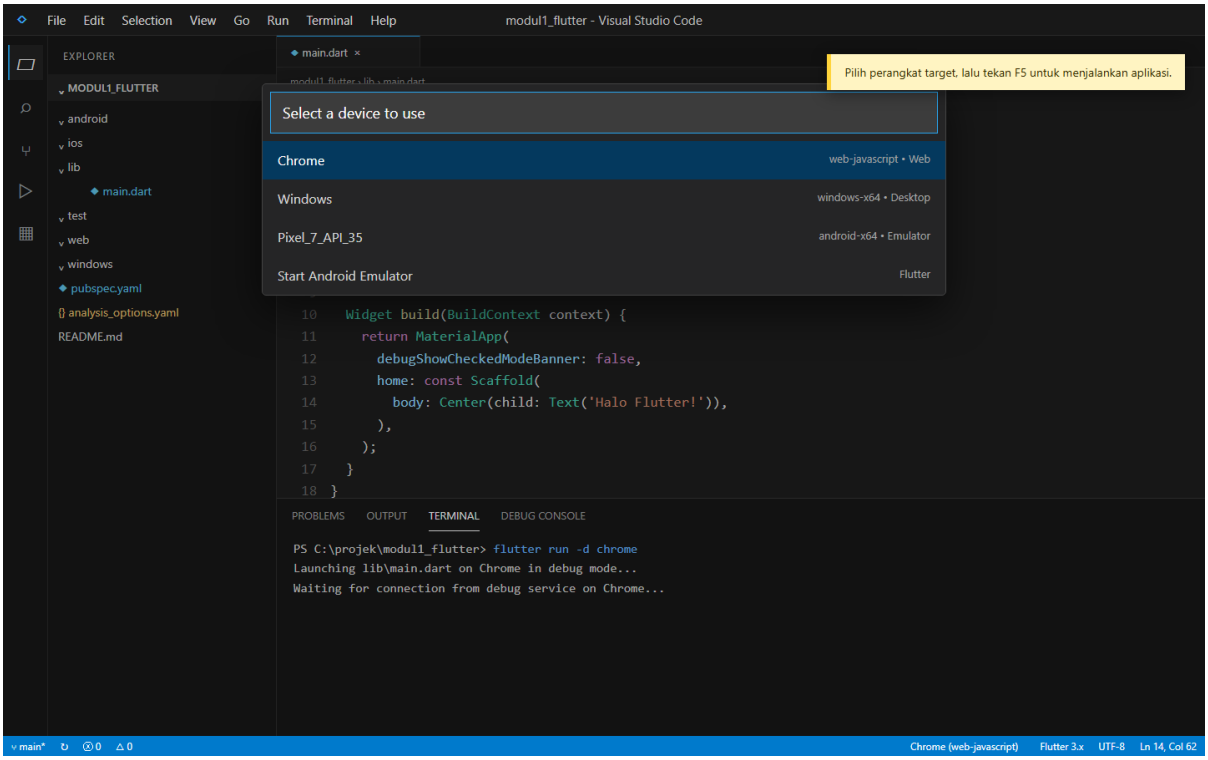
Gambar 34. Ilustrasi Command Palette untuk membuat proyek Flutter baru di Visual Studio Code

- Sebagai alternatif, proyek dapat dibuat melalui terminal dengan perintah berikut, kemudian buka foldernya melalui menu File → Open Folder.

```
flutter create modul1_flutter
cd modul1_flutter
code .
```

4. Memilih Perangkat dan Menjalankan Aplikasi

- Perhatikan Status Bar di sudut kanan bawah VS Code. Nama perangkat aktif tertera di sana, misalnya Chrome (web-javascript), Windows (windows-x64), atau nama emulator Android.
- Klik nama perangkat tersebut, atau buka Command Palette dan ketikkan Flutter: Select Device, kemudian pilih perangkat sasaran yang diinginkan.
- Apabila emulator Android belum berjalan, pilih Start Android emulator pada daftar perangkat, lalu tunggu hingga layar utama Android tampil.
- Jalankan aplikasi dalam mode debug dengan menekan tombol F5, atau melalui menu Run → Start Debugging.



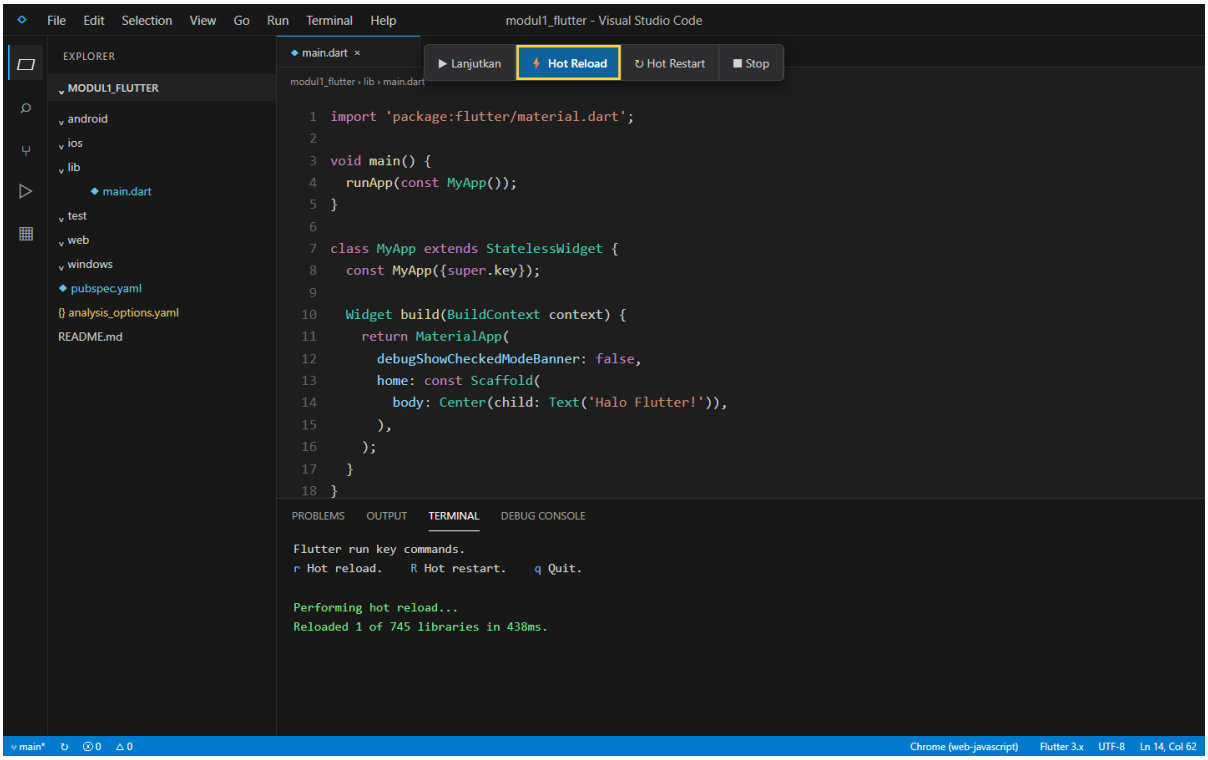
Gambar 35. Ilustrasi pemilihan perangkat target sebelum menjalankan aplikasi Flutter

- Aplikasi juga dapat dijalankan melalui terminal terintegrasi dengan menentukan perangkat sasaran secara langsung.

```
flutter run -d chrome
flutter run -d windows
flutter run -d emulator-5554
```

5. Menggunakan Hot Reload dan Hot Restart

- Simpan berkas dengan menekan Ctrl + S. Perubahan tampilan akan langsung terlihat pada perangkat tanpa memulai ulang aplikasi (hot reload).
- Hot reload juga dapat dijalankan melalui ikon kilat pada Debug Toolbar, atau dengan menekan tombol r pada terminal apabila aplikasi dijalankan melalui perintah flutter run.
- Gunakan hot restart (ikon panah melingkar, atau tombol R pada terminal) apabila perubahan menyangkut state awal aplikasi, variabel global, maupun berkas main.dart.

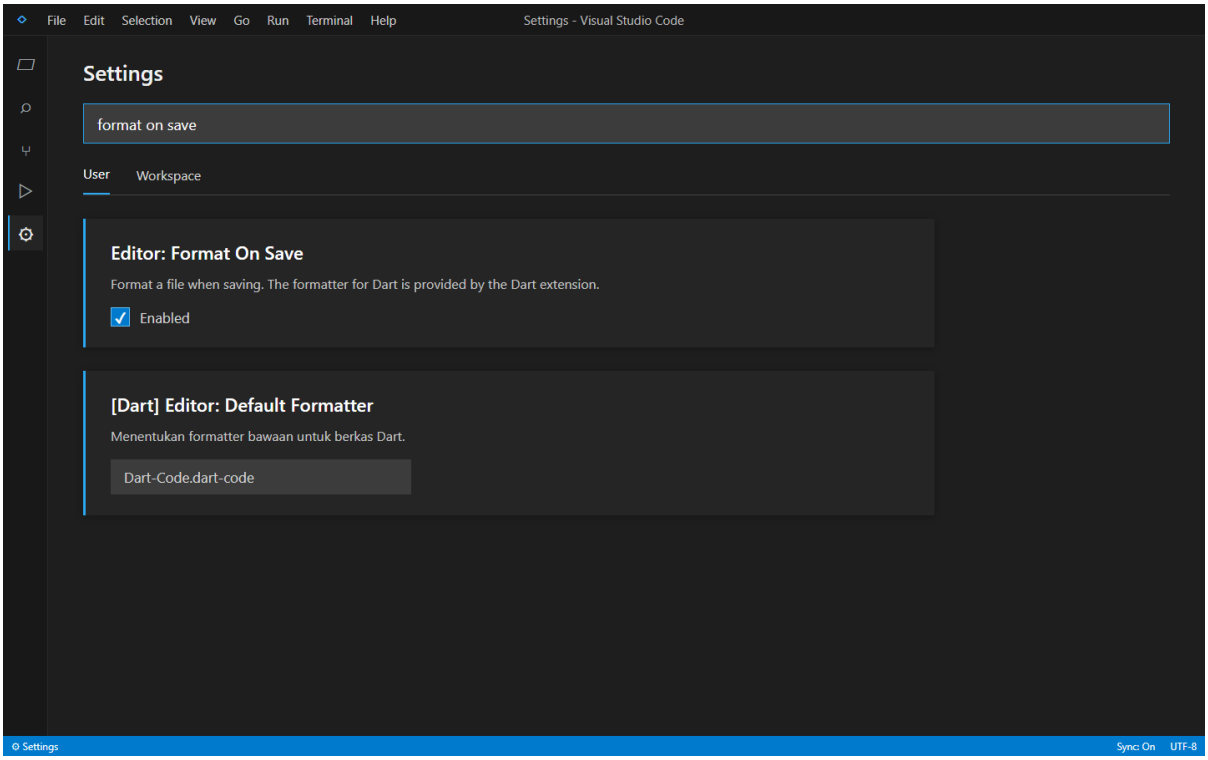


Gambar 36. Ilustrasi Hot Reload dan Hot Restart pada sesi debug Flutter

- Hentikan proses debug dengan menekan tombol Stop (ikon kotak merah) pada Debug Toolbar, atau tekan Shift + F5.

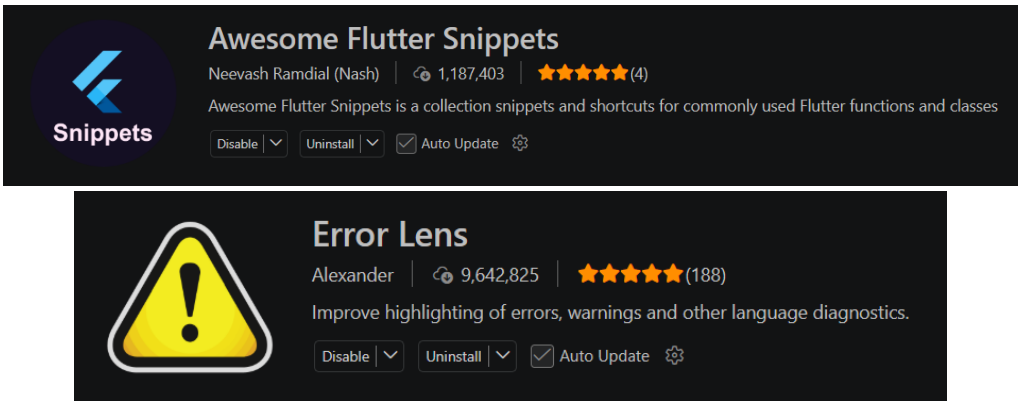
6. Pengaturan Tambahan yang Disarankan

- Aktifkan pemformatan kode otomatis melalui menu File → Preferences → Settings, kemudian centang Editor: Format On Save agar kode Dart tersusun rapi setiap kali disimpan.



Gambar 37. Ilustrasi pengaturan Editor: Format On Save untuk berkas Dart

- Pasang ekstensi pendukung seperti Awesome Flutter Snippets dan Error Lens untuk mempercepat penulisan kode dan mempermudah pembacaan pesan kesalahan.



Gambar 38. Snipets Awesome dan Error lens

- Gunakan menu View → Command Palette → Flutter: Run Flutter Doctor setiap kali terjadi galat pada lingkungan pengembangan, sebelum melakukan penyetelan ulang.

CATATAN Visual Studio Code hanya berperan sebagai editor kode. Proses kompilasi ke platform Android tetap membutuhkan Android SDK dan Android Emulator dari Android Studio, sedangkan target Windows membutuhkan Visual Studio 2022 dengan *workload Desktop development with* C++.

n. Sintax umum pada Flutter

Flutter menyediakan sejumlah perintah baris perintah (command line interface) yang digunakan selama proses pengembangan, mulai dari pembuatan proyek, menjalankan aplikasi, hingga membangun berkas rilis. Tabel berikut merangkum sintaks yang paling umum digunakan beserta fungsinya.

Tabel 1. Perintah dasar Flutter dan fungsinya

Perintah	Fungsi
flutter --version	Menampilkan versi Flutter dan Dart yang sedang terpasang.
flutter doctor	Memeriksa kelengkapan lingkungan pengembangan dan menampilkan komponen yang belum terpasang.
flutter doctor --android-licenses	Menyetujui lisensi Android SDK.
flutter upgrade	Memperbarui Flutter SDK ke versi terbaru pada channel yang aktif.
flutter create nama_proyek	Membuat proyek Flutter baru beserta struktur berkas standar.
flutter devices	Menampilkan daftar perangkat dan emulator yang terdeteksi.
flutter emulators	Menampilkan daftar emulator yang tersedia.
flutter emulators --launch <id>	Menjalankan emulator sesuai id yang dipilih.
flutter run	Menjalankan aplikasi pada perangkat aktif dalam mode debug.
flutter run -d chrome	Menjalankan aplikasi pada perangkat tertentu, misalnya Chrome atau emulator-5554.
flutter run --profile	Menjalankan aplikasi dalam mode profil untuk pengukuran kinerja.
flutter run --release	Menjalankan aplikasi dalam mode rilis tanpa alat bantu debug.

Perintah	Fungsi
flutter pub get	Mengunduh seluruh dependensi yang tercantum pada pubspec.yaml.
flutter pub upgrade	Memperbarui dependensi ke versi terbaru yang masih kompatibel.
flutter clean	Menghapus berkas hasil build dan cache proyek.
flutter build apk	Membangun berkas APK rilis untuk platform Android.
flutter build apk --split-per-abi	Membangun APK terpisah untuk setiap arsitektur agar ukurannya lebih kecil.
flutter build appbundle	Membangun berkas .aab untuk diunggah ke Google Play Store.
flutter build ios	Membangun aplikasi untuk platform iOS (hanya pada macOS).
flutter build web	Membangun aplikasi versi web pada direktori build/web.
flutter build windows	Membangun aplikasi desktop Windows (analog untuk linux dan macos).
r / R / q	Perintah pada konsol saat flutter run: hot reload, hot restart, dan keluar.
flutter build apk --debug	Membangun APK dalam mode debug untuk keperluan pengujian.
flutter build apk --flavor produksi	Membangun APK menggunakan konfigurasi flavor tertentu.
flutter build aar	Membangun modul Flutter menjadi berkas .aar untuk proyek Android native.
flutter build ipa	Membangun berkas .ipa untuk distribusi ke App Store (hanya pada macOS).
flutter build bundle	Membangun berkas aset dan kode Dart tanpa mengemasnya ke installer.
flutter build web --release	Membangun aplikasi web versi rilis dengan optimasi penuh.
flutter build linux	Membangun aplikasi desktop Linux.
flutter build macos	Membangun aplikasi desktop macOS (hanya pada macOS).
flutter clean && flutter pub get	Kombinasi yang umum digunakan untuk memulihkan proyek yang bermasalah.

F. Praktikum

a. Praktikum: *Stateless Widget, Stateful Widget,* serta *Layout dan Styling*

Pada bagian ini mahasiswa membangun satu proyek Flutter bernama **praktikum_widget**. Proyek tersebut digunakan untuk meriview materi Dasar Pemrograman Perangkat Bergerak semester 3, dalam hal ini mempelajari tiga materi sekaligus, yaitu *Stateless Widget* sebagai komponen tampilan statis, *Stateful Widget* sebagai komponen yang dapat berubah sesuai interaksi pengguna, serta penyusunan layout dan styling agar tampilan aplikasi rapi dan konsisten. Ikuti seluruh langkah secara berurutan.

1. Membuat Proyek Praktikum

- Buka Visual Studio Code, kemudian buka terminal melalui menu Terminal > New Terminal atau tekan **Ctrl +** `.

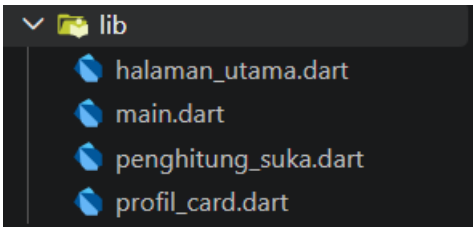
- Pindah ke direktori kerja yang diinginkan, misalnya direktori Documents, lalu jalankan perintah pembuatan proyek berikut.

```
flutter create praktikum_widget
cd praktikum_widget
code .
```

- Tunggu hingga proses pembuatan proyek selesai. Struktur direktori proyek akan muncul pada panel Explorer VS Code.

2. Menyiapkan Struktur Berkas Proyek

- Perhatikan folder lib pada panel Explorer. Secara bawaan folder tersebut hanya berisi berkas main.dart.
- Klik kanan pada folder lib, pilih New File, kemudian buat tiga berkas baru bernama **profil_card.dart, penghitung_suka.dart,** dan **halaman_utama.dart.**
- Pemisahan berkas ini bertujuan agar setiap widget berada pada berkas tersendiri sehingga kode lebih mudah dibaca dan dirawat.
- Struktur akhir folder lib adalah sebagai berikut.



Gambar 39. Struktur berkas proyek praktikum_widget pada folder lib

3. Membuat Stateless Widget

- StatelessWidget adalah widget yang tampilannya tidak berubah setelah dibangun. Widget jenis ini digunakan untuk menampilkan data yang bersifat tetap, misalnya kartu identitas mahasiswa.
- Buka berkas profil_card.dart, kemudian tuliskan kode berikut secara lengkap.

```
import 'package:flutter/material.dart';
class ProfilCard extends StatelessWidget {
  const ProfilCard({
    super.key,
    required this.nama,
    required this.nim,
    required this.prodi,
  });

  final String nama;
  final String nim;
  final String prodi;

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
```

```
        color: Colors.white,
        borderRadius: BorderRadius.circular(12),
        boxShadow: const [
          BoxShadow(
            color: Colors.black12,
            blurRadius: 6,
            offset: Offset(0, 3),
          ),
        ],
      ),
      child: Row(
        children: [
          const CircleAvatar(
            radius: 28,
            backgroundColor: Colors.indigo,
            child: Icon(Icons.person, color: Colors.white,
size: 28),
          ),
          const SizedBox(width: 16),
          Expanded(
            child: Column(
              crossAxisAlignment:
CrossAxisAlignment.start,
              children: [
                Text(
                  nama,
                  style: const TextStyle(
                    fontSize: 18,
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 4),
                Text(
                  'NIM: $nim',
                  style: const TextStyle(fontSize: 14, color:
Colors.black54),
                ),
                Text(
                  prodi,
                  style: const TextStyle(fontSize: 14, color:
Colors.black54),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

- Perhatikan bahwa **seluruh nilai (nama, nim, dan prodi)** diterima melalui **konstruktor** dan disimpan pada **variabel final**. Nilai tersebut **tidak dapat diubah** setelah widget dibangun, inilah ciri utama **StatelessWidget**.
- Simpan berkas dengan menekan Ctrl + S. Apabila terdapat garis merah pada editor, periksa kembali tanda kurung dan titik koma.

4. Membuat Stateful Widget

- StatefulWidget adalah widget yang tampilannya dapat berubah selama aplikasi berjalan. Perubahan dilakukan melalui fungsi setState() yang memberi tahu Flutter agar membangun ulang bagian tampilan terkait.
- Buka berkas penghitung_suka.dart, kemudian tuliskan kode berikut.

```
import 'package:flutter/material.dart';
class PenghitungSuka extends StatefulWidget {
  const PenghitungSuka({super.key});

  @override
  State<PenghitungSuka> createState() => _PenghitungSukaState();
}

class _PenghitungSukaState extends State<PenghitungSuka> {
  int _jumlahSuka = 0;
  bool _disukai = false;

  void _tambahSuka() {
    setState(() {
      _jumlahSuka++;
      _disukai = true;
    });
  }

  void _resetSuka() {
    setState(() {
      _jumlahSuka = 0;
      _disukai = false;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Card(
      elevation: 2,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
      ),
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
```

```
          children: [
            Text(
              'Jumlah Suka: $_jumlahSuka',
              style: const TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.w600,
              ),
            ),
            const SizedBox(height: 12),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                ElevatedButton.icon(
                  onPressed: _tambahSuka,
                  icon: Icon(
                    _disukai ? Icons.favorite : Icons.favorite_border,
                  ),
                  label: const Text('Suka'),
                ),
                const SizedBox(width: 12),
                OutlinedButton(
                  onPressed: _resetSuka,
                  child: const Text('Reset'),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

- Variabel **_jumlahSuka** dan **_disukai** merupakan **state**, yaitu data yang dapat berubah. Setiap kali tombol Suka ditekan, nilai **_jumlahSuka** bertambah dan ikon hati berubah menjadi terisi.
- Perubahan nilai wajib dilakukan di dalam **setState()**. Apabila nilai diubah tanpa **setState()**, data memang berubah tetapi tampilan tidak diperbarui.
- Simpan berkas dengan menekan Ctrl + S.

5. Menyusun Layout dan Styling

- **Layout** mengatur posisi widget, sedangkan **styling** mengatur tampilan visual seperti warna, sudut, jarak, dan huruf. Pada langkah ini kedua widget yang telah dibuat digabungkan menjadi satu halaman.
- Buka berkas halaman_utama.dart, kemudian tuliskan kode berikut.

```
import 'package:flutter/material.dart';
import 'profil_card.dart';
import 'penghitung_suka.dart';
```

```
class HalamanUtama extends StatelessWidget {
  const HalamanUtama({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color(0xFFF2F4F8),
      appBar: AppBar(
        title: const Text('Praktikum Widget Flutter'),
        backgroundColor: Colors.indigo,
        foregroundColor: Colors.white,
        centerTitle: true,
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            const ProfilCard(
              nama: 'Nama Mahasiswa',
              nim: '2211001',
              prodi: 'Sistem Informasi Kota Cerdas',
            ),
            const SizedBox(height: 20),
            const Text(
              'Statistik Interaksi',
              style: TextStyle(
                fontSize: 16,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 8),
            const PenghitungSuka(),
            const SizedBox(height: 20),
            Row(
              children: const [
                Expanded(
                  child: KotakInfo(
                    judul: 'Widget',
                    nilai: '3',
                    warna: Colors.indigo,
                  ),
                ),
                SizedBox(width: 12),
                Expanded(
                  child: KotakInfo(
                    judul: 'Berkas',
                    nilai: '4',
                    warna: Colors.teal,
```

```
                ),
              ),
            ],
          ),
        ],
      ),
    ),
  );
 }
}

class KotakInfo extends StatelessWidget {
 const KotakInfo({
  super.key,
  required this.judul,
  required this.nilai,
  required this.warna,
 });

 final String judul;
 final String nilai;
 final Color warna;

 @override
 Widget build(BuildContext context) {
  return Container(
   padding: const EdgeInsets.symmetric(vertical: 20),
   alignment: Alignment.center,
   decoration: BoxDecoration(
    color: warna.withOpacity(0.1),
    border: Border.all(color: warna),
    borderRadius: BorderRadius.circular(10),
   ),
   child: Column(
    children: [
     Text(
      nilai,
      style: TextStyle(
       fontSize: 22,
       fontWeight: FontWeight.bold,
       color: warna,
      ),
     ),
     const SizedBox(height: 4),
     Text(
      judul,
      style: TextStyle(fontSize: 13, color: warna),
     ),
    ],
   ),
```

```
    );
  }
}
```

- Simpan berkas, kemudian perhatikan fungsi setiap **widget layout** dan **styling** yang digunakan pada tabel berikut.

Tabel 2. Widget dasar untuk layout dan styling beserta fungsinya

Widget	Fungsi
Scaffold	Menyediakan kerangka dasar halaman: AppBar, body, dan latar belakang.
AppBar	Bilah judul di bagian atas halaman.
Column	Menyusun widget anak secara vertikal (atas ke bawah).
Row	Menyusun widget anak secara horizontal (kiri ke kanan).
Expanded	Membuat widget anak mengisi ruang sisa yang tersedia di dalam Row atau Column.
SizedBox	Memberi jarak tetap atau membatasi ukuran widget.
Container	Wadah serbaguna yang dapat diberi ukuran, warna, jarak, dan dekorasi.
Padding / EdgeInsets	Memberi jarak di dalam widget terhadap tepinya.
SingleChildScrollView	Membuat isi halaman dapat digulir apabila melebihi tinggi layar.
Card	Kotak dengan sudut membulat dan efek bayangan bawaan.
BoxDecoration	Mengatur warna, garis tepi, sudut membulat, dan bayangan pada Container.
BorderRadius.circular	Membuat sudut widget menjadi membulat.
TextStyle	Mengatur ukuran, ketebalan, dan warna teks.
CrossAxisAlignment	Mengatur perataan widget anak pada sumbu silang.
MainAxisAlignment	Mengatur perataan widget anak pada sumbu utama.

6. Menyatukan Aplikasi pada main.dart

- Buka berkas main.dart, hapus seluruh kode bawaan, kemudian tuliskan kode berikut.

```
import 'package:flutter/material.dart';
import 'halaman_utama.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
      title: 'Praktikum Widget',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        useMaterial3: true,
      ),
      home: const HalamanUtama(),
    );
  }
}
```

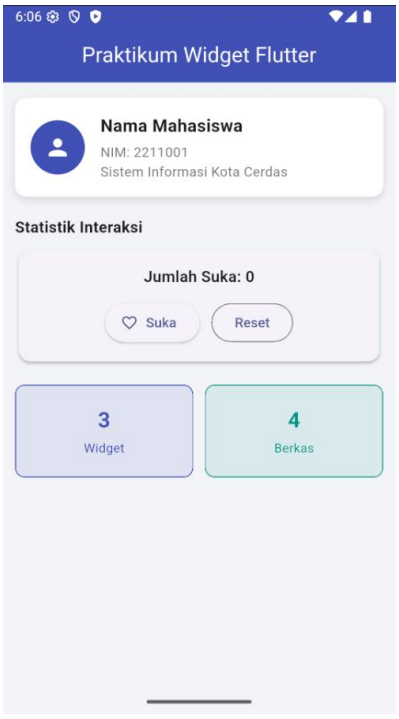
- **Fungsi main()** merupakan titik awal aplikasi, sedangkan **runApp()** menampilkan widget pertama ke layar. **MaterialApp** menyediakan tema, judul, dan navigasi bawaan Material Design.

7. Menjalankan dan Menguji Aplikasi

- Pastikan emulator atau perangkat sudah aktif, kemudian periksa daftar perangkat dan jalankan aplikasi melalui terminal.

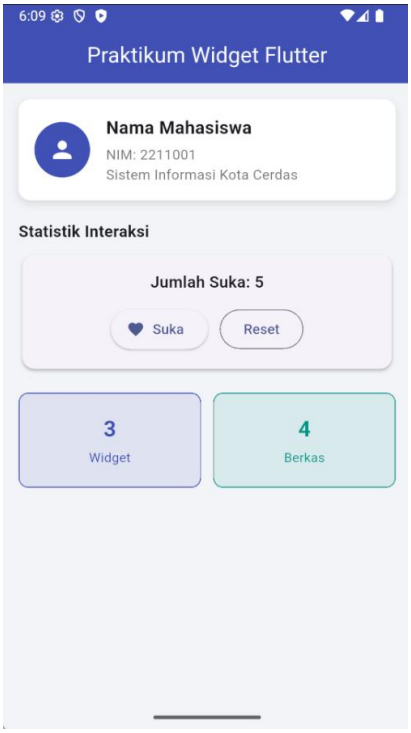
```
flutter devices
flutter run
```

- Tunggu hingga proses kompilasi selesai. Tampilan awal aplikasi akan terlihat seperti gambar berikut.



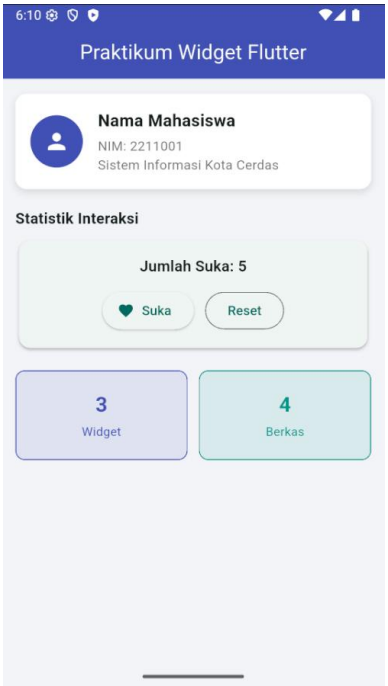
Gambar 40. Tampilan awal aplikasi praktikum_widget

- Tekan tombol Suka beberapa kali. Angka pada penghitung akan bertambah dan ikon hati berubah menjadi terisi, sedangkan kartu profil tetap sama.



Gambar 41. Perubahan tampilan akibat setState() pada Stateful Widget

- Selanjutnya, ubah nilai backgroundColor pada AppBar dari Colors.indigo menjadi Colors.teal, lalu simpan berkas. Perubahan warna akan langsung terlihat melalui hot reload tanpa memulai ulang aplikasi.



Gambar 42. Hasil perubahan styling melalui hot reload

- Dari percobaan tersebut dapat disimpulkan bahwa **ProfilCard** sebagai **Stateless Widget** hanya berubah apabila kodenya diubah, sedangkan **PenghitungSuka** sebagai **Stateful Widget** dapat berubah saat aplikasi berjalan melalui interaksi pengguna.

b. Praktikum Berbantuan AI: Membangun Ulang Aplikasi Widget melalui Prompt Bertahap

Bagian ini menggunakan studi kasus yang sama dengan bagian sebelumnya, yaitu aplikasi **praktikum_widget** yang terdiri atas **Stateless Widget, Stateful Widget**, serta **layout** dan **styling**. Perbedaannya, kode tidak langsung disalin dari modul, melainkan dihasilkan melalui percakapan dengan asisten AI. Tujuannya bukan agar mahasiswa bergantung pada AI, tetapi agar mahasiswa mampu menyusun prompt yang tepat, membaca kode yang dihasilkan, memverifikasi kebenarannya, dan menjelaskan setiap barisnya. Kode dari AI hanya diterima apabila mahasiswa memahami cara kerjanya. Aturan penting selama praktikum ini: setiap kali AI memberikan kode, mahasiswa wajib membaca ulang kode tersebut, menjalankannya, dan mencatat pada laporan bagian mana yang belum dipahami beserta hasil penelusurannya.

1. Buat Projek Flutter

- Siapkan proyek yang akan dikerjakan melalui terminal VS Code.

```
flutter create praktikum_widget_ai
cd praktikum_widget_ai
code .
```

- Proyek baru sengaja dibuat terpisah agar hasil pengerjaan mandiri pada bagian sebelumnya tetap utuh dan dapat dibandingkan.

2. Menyusun Prompt Konteks Awal

- Prompt yang baik tidak berbunyi "buatkan aplikasi Flutter", melainkan memuat peran, konteks, tugas, batasan, dan format keluaran. Perhatikan unsur prompt pada tabel berikut sebelum menulis prompt pertama.

Tabel 3. Unsur penyusun prompt dan penjelasannya

Unsur Prompt	Penjelasan
Peran	Menetapkan posisi AI, misalnya sebagai asisten pemrograman Flutter untuk pemula.
Konteks	Menjelaskan nama proyek, materi yang sedang dipelajari, dan tingkat kemampuan mahasiswa.
Tugas	Menyatakan satu pekerjaan yang jelas, misalnya membuat satu berkas widget tertentu.
Batasan	Membatasi paket, gaya kode, dan pendekatan agar hasilnya sesuai materi kuliah.
Format keluaran	Menentukan bentuk jawaban, misalnya rencana kerja dahulu, kode kemudian, disertai penjelasan.
Verifikasi	Meminta AI menjelaskan kode agar mahasiswa dapat menguji pemahamannya sendiri.

- Ketik prompt pembuka berikut pada panel percakapan AI, kemudian tekan Enter.

```
Kamu berperan sebagai asisten pemrograman Flutter untuk mahasiswa
yang baru belajar widget.

Konteks: saya mengerjakan proyek Flutter bernama praktikum_widget_ai
```

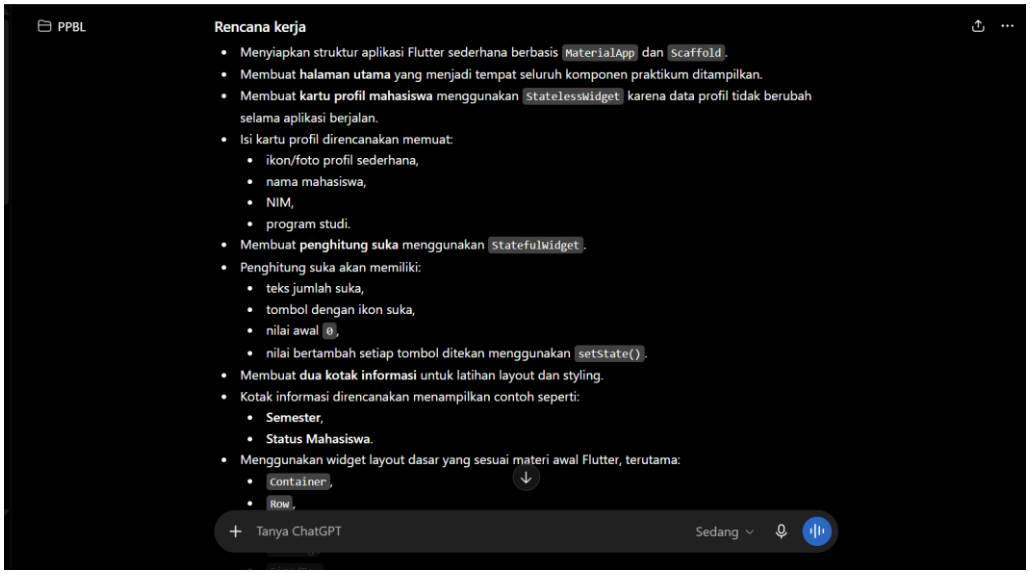
```
pada Visual Studio Code. Materi yang sedang dipelajari adalah
Stateless Widget, Stateful Widget, serta layout dan styling.

Tugas: bantu saya membangun satu halaman aplikasi yang memuat
kartu profil mahasiswa, penghitung suka, dan dua kotak informasi.

Batasan: gunakan hanya package:flutter/material.dart, tanpa paket
tambahan dan tanpa state management eksternal.

Format keluaran: tuliskan dahulu rencana kerja dalam bentuk poin
beserta daftar berkas yang akan dibuat. Jangan menulis kode apa pun
sebelum saya meminta berkas tertentu.
```

- AI akan menjawab dengan rencana kerja dan daftar berkas. Bandingkan rencana tersebut dengan struktur yang telah dipelajari pada bagian sebelumnya, yaitu profil_card.dart, penghitung_suka.dart, halaman_utama.dart, dan main.dart.
- Apabila rencana AI berbeda, jangan langsung menerimanya. Tanyakan alasannya dengan prompt lanjutan, misalnya: "Mengapa kamu memisahkan berkas seperti itu? Bandingkan dengan pemisahan satu widget satu berkas."



Gambar 43. Balasan asisten AI berupa rencana kerja sebelum menulis kode

3. Prompt untuk Stateless Widget

- Buat berkas kosong lib/profil_card.dart terlebih dahulu melalui klik kanan pada folder lib, lalu pilih New File. Berkas dibuat sendiri oleh mahasiswa agar struktur proyek tetap dipahami.
- Kirim prompt berikut pada panel percakapan AI.

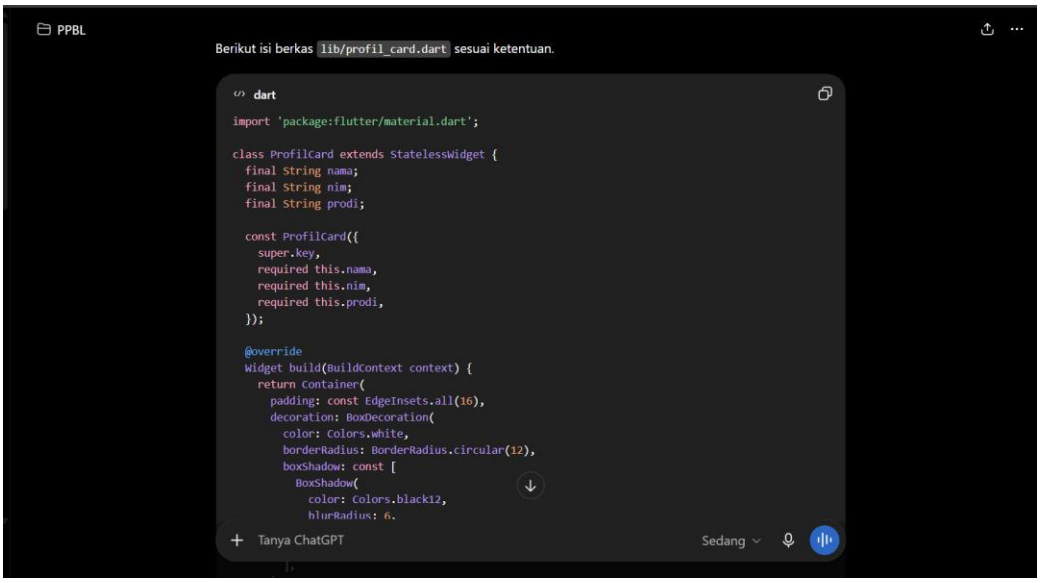
```
Buatkan isi berkas lib/profil_card.dart.

Ketentuan:
1. Satu StatelessWidget bernama ProfilCard.
2. Parameter wajib: nama, nim, prodi, semuanya bertipe String.
3. Gunakan const constructor dan super.key.
```

```
4. Tampilan: Container dengan padding 16, sudut membulat 12,
   warna putih, dan bayangan tipis.
5. Di dalamnya Row berisi CircleAvatar dengan ikon person, lalu
   Column berisi nama (ukuran 18, tebal), nim, dan prodi
   (ukuran 14, warna abu-abu).
6. Hanya gunakan package:flutter/material.dart.

Setelah kode, jelaskan fungsi setiap widget dalam satu kalimat
per widget, memakai bahasa Indonesia.
```

- Salin kode hasil AI ke dalam berkas profil_card.dart, kemudian simpan dengan Ctrl + S. Perhatikan apakah muncul garis merah pada editor.
- Sebelum melanjutkan, lakukan verifikasi berikut secara mandiri: class benar-benar merupakan turunan StatelessWidget; ketiga parameter ditandai required; tidak ada import selain material.dart; dan tidak ada fungsi setState() karena widget ini tidak menyimpan state.
- Apabila salah satu poin verifikasi tidak terpenuhi, jangan memperbaiki secara manual terlebih dahulu. Kirim prompt perbaikan agar Saudara terbiasa mengarahkan AI, misalnya: "Kode kamu masih memakai paket tambahan. Tulis ulang hanya dengan material.dart dan jelaskan perubahannya."



Gambar 44. Hasil prompt Stateless Widget beserta penjelasannya di VS Code

4. Prompt untuk Stateful Widget

- Buat berkas lib/penghitung_suka.dart, kemudian kirim prompt berikut.

```
Buatkan isi berkas lib/penghitung_suka.dart.

Ketentuan:
1. Satu StatefulWidget bernama PenghitungSuka.
2. Simpan dua state: _jumlahSuka bertipe int dan _disukai bertipe bool.
3. Sediakan tombol Suka untuk menambah _jumlahSuka dan mengubah
   ikon hati menjadi terisi, serta tombol Reset untuk mengembalikan
   nilai ke kondisi awal.
```

```
4. Seluruh perubahan nilai harus dilakukan di dalam setState().
5. Bungkus tampilan dengan Card bersudut membulat 12 dan padding 16.
6. Hanya gunakan package:flutter/material.dart.

Setelah kode, jelaskan dalam bahasa Indonesia: apa yang terjadi
apabila nilai _jumlahSuka diubah tanpa setState().
```

- Bacalah penjelasan AI mengenai setState(), kemudian uji pemahaman dengan percobaan kecil: hapus sementara pembungkus setState() pada fungsi penambah, jalankan aplikasi, lalu amati bahwa angka pada layar tidak berubah meskipun nilainya bertambah. Kembalikan kode setelah percobaan selesai dan catat hasilnya pada laporan.
- Ajukan pula prompt pendalaman agar pemahaman tidak berhenti pada kode, misalnya: "Jelaskan perbedaan StatelessWidget dan StatefulWidget menggunakan analogi sederhana, lalu sebutkan satu contoh kasus pada aplikasi ini untuk masing-masing."

5. Prompt untuk Layout dan Styling

- Buat berkas lib/halaman_utama.dart, kemudian kirim prompt berikut.

```
Buatkan isi berkas lib/halaman_utama.dart.

Ketentuan:
1. StatelessWidget bernama HalamanUtama yang mengembalikan Scaffold.
2. AppBar berjudul 'Praktikum Widget Flutter' dengan warna indigo,
   teks putih, dan judul di tengah.
3. Body berupa SingleChildScrollView dengan padding 16 dan Column
   di dalamnya.
4. Susunan isi Column: ProfilCard, jarak 20, judul 'Statistik
   Interaksi', jarak 8, PenghitungSuka, jarak 20, lalu Row berisi
   dua kotak informasi yang masing-masing memakai Expanded.
5. Buat widget KotakInfo terpisah pada berkas yang sama, berisi
   Container dengan warna transparan tipis, garis tepi, dan sudut
   membulat 10.
6. Impor profil_card.dart dan penghitung_suka.dart.

Setelah kode, buat tabel dua kolom berisi nama widget layout yang
kamu pakai dan fungsinya, dalam bahasa Indonesia.
```

- Simpan berkas, lalu cocokkan tabel dari AI dengan tabel widget layout pada bagian sebelumnya. Apabila terdapat widget yang belum dikenali, ajukan prompt lanjutan, misalnya: "Jelaskan perbedaan Expanded dan Flexible beserta contoh singkat pada kode di atas."
- Latih pula kemampuan mengubah tampilan melalui prompt, misalnya: "Ubah tata letak dua kotak informasi dari Row menjadi Column, tampilkan hanya bagian kode yang berubah, dan jelaskan dampaknya pada tampilan." Kebiasaan meminta hanya bagian yang berubah memudahkan mahasiswa menelusuri perubahan kode.

6. Prompt untuk Menyatukan Aplikasi pada main.dart

- Buka berkas lib/main.dart yang sudah ada, kemudian kirim prompt berikut.

```
Tulis ulang isi berkas lib/main.dart.

Ketentuan:
1. Fungsi main() memanggil runApp().
2. Widget MyApp mengembalikan MaterialApp dengan judul
   'Praktikum Widget', banner debug dinonaktifkan, tema
   colorSchemeSeed indigo, dan useMaterial3 aktif.
3. Properti home diarahkan ke HalamanUtama.
4. Impor halaman_utama.dart.

Jelaskan perbedaan peran main(), runApp(), dan MaterialApp
dalam tiga poin singkat.
```

- Setelah kode disalin dan disimpan, mahasiswa wajib dapat menjawab pertanyaan berikut tanpa bantuan AI: berkas mana yang dijalankan pertama, widget mana yang tampil pertama, dan dari mana warna tema aplikasi berasal.

7. Prompt untuk Menelusuri dan Memperbaiki Galat

- Jalankan aplikasi melalui terminal atau tekan F5. Apabila muncul galat, jangan langsung menyalin seluruh pesan galat tanpa membacanya.

```
flutter run
```

- Bacalah pesan galat terlebih dahulu, tandai nama berkas dan nomor barisnya, kemudian gunakan prompt penelusuran berikut dengan menempelkan pesan galat pada bagian yang ditandai.

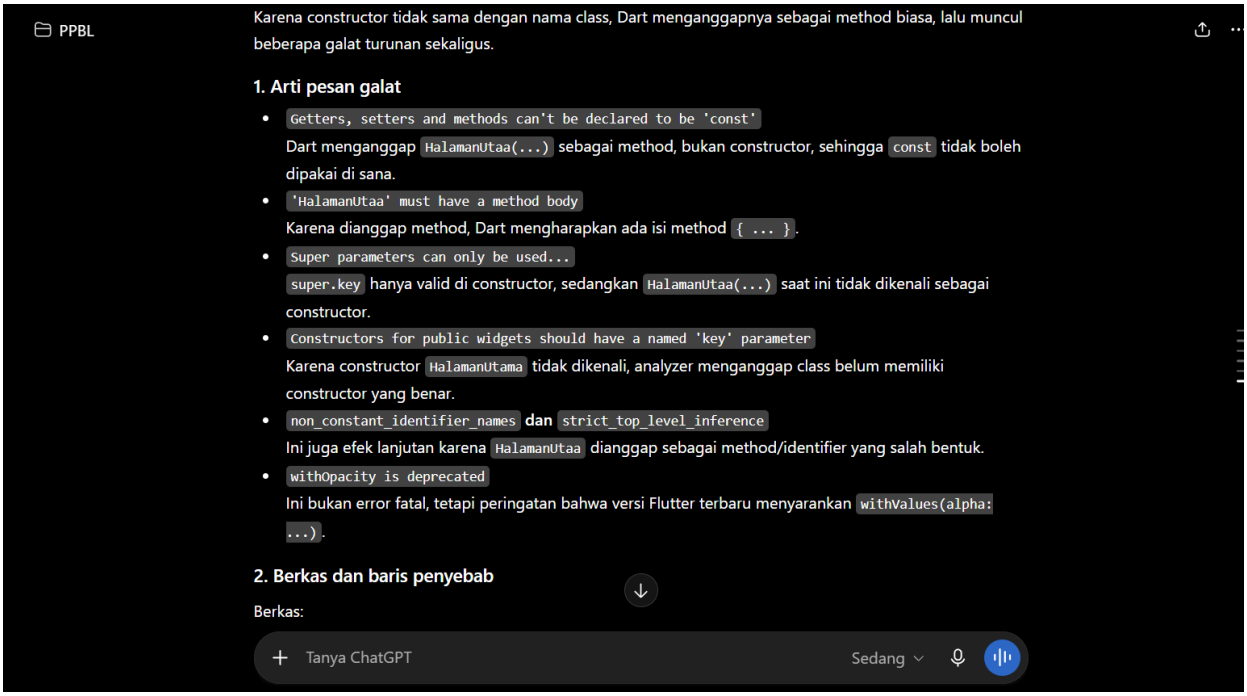
```
Saya mendapatkan galat berikut saat menjalankan aplikasi Flutter:

<tempelkan pesan galat di sini>

Tugas kamu:
1. Jelaskan arti pesan galat tersebut dengan bahasa sederhana.
2. Sebutkan berkas dan baris yang menjadi penyebabnya.
3. Berikan perbaikan minimal, hanya bagian kode yang berubah.
4. Jelaskan mengapa perbaikan itu menyelesaikan masalah.

Jangan menulis ulang seluruh berkas.
```

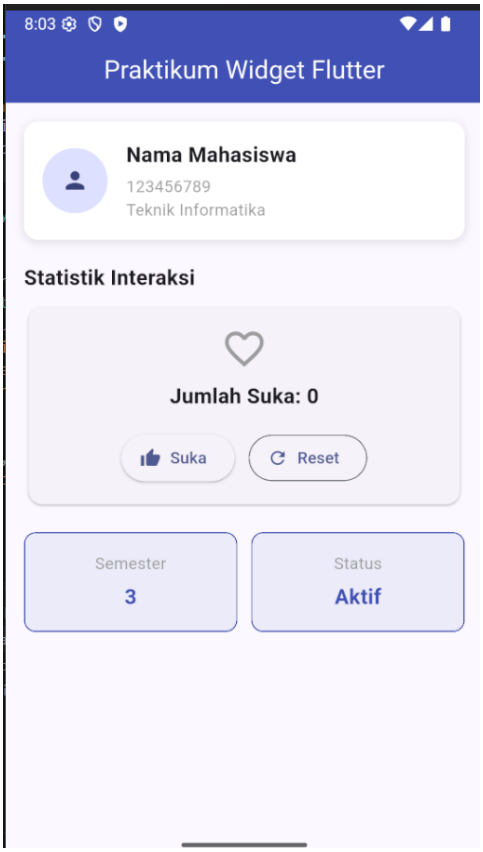
- Terapkan perbaikan secara manual, bukan dengan menyalin seluruh berkas. Cara ini memastikan Saudara mengetahui bagian kode mana yang berubah dan mengapa.



Gambar 45. Penelusuran galat menggunakan bantuan asisten AI

8. Menguji Aplikasi dan Menguji Pemahaman

- Jalankan kembali aplikasi hingga tampil tanpa galat, kemudian tekan tombol Suka beberapa kali dan tombol Reset untuk memastikan state berfungsi.
- Lakukan pula percobaan hot reload dengan mengubah warna AppBar menjadi Colors.teal, lalu simpan berkas dan amati perubahannya tanpa memulai ulang aplikasi.



Gambar 46. Aplikasi hasil praktikum berbantuan AI yang berjalan dari Visual Studio Code

- Setelah aplikasi berjalan, tutup panel percakapan AI dan jawab pertanyaan berikut secara mandiri sebagai bukti pemahaman: widget mana yang bersifat stateless dan mengapa; apa yang dilakukan setState(); widget apa yang mengatur susunan vertikal dan horizontal; serta bagian kode mana yang mengatur warna, jarak, dan sudut membulat.
- Apabila terdapat pertanyaan yang belum dapat dijawab, tandai bagian kode terkait dan gunakan prompt penjelasan baris demi baris berikut.

```
Jelaskan kode berikut baris demi baris untuk mahasiswa pemula,
tanpa mengubah kodenya:

<tempelkan potongan kode yang belum dipahami>

Akhiri dengan satu pertanyaan singkat untuk menguji apakah saya
sudah benar-benar memahami potongan kode tersebut.
```

- Rambu penggunaan AI selama praktikum: kode yang tidak dapat dijelaskan tidak boleh dimasukkan ke laporan; setiap prompt yang digunakan wajib dilampirkan pada laporan; dan hasil AI yang keliru harus dicatat sebagai temuan, bukan dihapus, karena melatih ketelitian membaca kode.

G. Tugas Praktikum

Soal: Dasbor Pilar Smart City "Kota Nusantara Cerdas"

Pemerintah Kota Nusantara sedang merintis program transformasi menuju smart city. Dinas Komunikasi dan Informatika membentuk tim pengembang muda yang bertugas membuat purwarupa aplikasi bergerak bernama Nusantara Cerdas. Aplikasi ini akan dipasang pada layar informasi publik di balai kota agar warga dapat memantau perkembangan enam pilar smart city, yaitu Smart Governance, Smart Economy, Smart Living, Smart Mobility, Smart Environment, dan Smart People.

Kepala dinas menjelaskan bahwa halaman utama aplikasi harus menampilkan identitas program berupa nama kota, semboyan kota, dan logo atau ikon kota. Bagian ini bersifat tetap karena tidak pernah berubah selama aplikasi berjalan. Di bawahnya, warga ingin melihat kartu ringkasan untuk setiap pilar smart city yang memuat nama pilar, ikon yang mewakili pilar tersebut, dan deskripsi singkat satu kalimat mengenai layanan yang diberikan.

Selain informasi yang bersifat tetap, aplikasi juga harus menampilkan data yang berubah. Petugas balai kota membutuhkan panel Laporan Warga yang mencatat jumlah laporan masuk pada hari berjalan. Setiap kali petugas menekan tombol Laporan Masuk, jumlah laporan bertambah satu. Ketika laporan selesai ditangani dan tombol Laporan Selesai ditekan, jumlahnya berkurang satu dan tidak boleh kurang dari nol. Tersedia pula tombol Reset Harian untuk mengembalikan hitungan ke angka nol pada awal hari. Status pelayanan yang tampil ikut menyesuaikan jumlah laporan, misalnya Pelayanan Lancar ketika laporan kurang dari lima, Pelayanan Sibuk ketika laporan berada pada rentang lima sampai sepuluh, dan Perlu Penambahan Petugas ketika laporan melebihi sepuluh.

Kepala dinas menegaskan bahwa tampilan aplikasi harus rapi dan nyaman dibaca dari jarak jauh, menggunakan tema warna kota, jarak antar bagian yang teratur, serta tetap dapat digulir apabila layar perangkat berukuran kecil. Sebagai anggota tim pengembang, tugas Anda adalah mewujudkan purwarupa aplikasi tersebut menggunakan Flutter.

Ketentuan Pengerjaan

- Buat proyek Flutter baru dengan nama nusantara_cerdas_nim
- Buat stateless widget bernama KartuPilar pada berkas kartu_pilar.dart yang menerima parameter nama pilar, ikon, dan deskripsi singkat, kemudian gunakan widget tersebut untuk menampilkan keenam pilar smart city.
- Buat stateless widget bernama KepalaKota pada berkas kepala_kota.dart untuk menampilkan nama kota, semboyan, dan logo atau ikon kota.
- Buat stateful widget bernama PanelLaporanWarga pada berkas panel_laporan.dart yang memuat tombol Laporan Masuk, Laporan Selesai, dan Reset Harian, serta teks status pelayanan yang berubah mengikuti jumlah laporan. Gunakan setState untuk memperbarui tampilan dan pastikan nilai penghitung tidak pernah negatif.
- Susun seluruh widget pada berkas halaman_utama.dart menggunakan Scaffold, AppBar berjudul Nusantara Cerdas, serta SingleChildScrollView agar tampilan tetap rapi pada layar kecil.
- Terapkan layout dan styling menggunakan Column, Row, Container atau Card, Padding, dan SizedBox, dengan tema warna yang konsisten melalui ThemeData pada berkas main.dart.
- Jalankan aplikasi pada emulator atau perangkat fisik, lalu ambil tangkapan layar tampilan awal dan tampilan setelah penghitung laporan diubah.

Luaran yang Dikumpulkan

- Kode program lengkap yang diunggah ke repositori Git.
- Laporan dalam format PDF, dan dikumpulkan di LMS.
- Penjelasan singkat pada laporan mengenai alasan pemilihan stateless widget untuk bagian identitas dan kartu pilar, serta stateful widget untuk panel laporan warga.
- Berkas laporan diberi nama Modul1_NIM_NamaLengkap.pdf dan diunggah melalui LMS sesuai tenggat yang ditentukan.

H. Format Pengumpulan Tugas

Laporan disusun secara berurutan dengan sistematika berikut.

1. **Halaman Sampul (Cover)**
   Memuat judul praktikum, nama mahasiswa, NIM, kelas, mata kuliah, nama dosen pengampu, program studi, serta tahun akademik. Gunakan logo institusi dan tata letak yang rapi.
2. **Tujuan Praktikum**
   Menuliskan tujuan yang ingin dicapai pada praktikum ini dengan kalimat sendiri, bukan menyalin ulang tujuan pembelajaran pada modul.
3. **Alat dan Bahan**
   Menyebutkan spesifikasi perangkat keras yang digunakan serta perangkat lunak beserta versinya, misalnya Flutter SDK, Dart SDK, Android Studio, Visual Studio Code, dan Git.
4. **Langkah-Langkah Praktikum**
   Menguraikan tahapan pengerjaan secara runtut mulai dari pembuatan proyek, pembuatan stateless widget, stateful widget, penyusunan layout dan styling, hingga penggabungan pada main.dart. Setiap langkah disertai potongan kode yang relevan dan penjelasan singkat.
5. **Hasil dan Dokumentasi Program**

Menampilkan tangkapan layar hasil aplikasi yang berjalan, sekurang-kurangnya tampilan awal dan tampilan setelah terjadi perubahan state. Setiap gambar diberi nomor, keterangan gambar, dan penjelasan singkat.

6. **Analisis dan Pembahasan**
   Membahas alasan pemilihan stateless widget dan stateful widget pada masing-masing bagian aplikasi, kendala atau galat yang ditemukan beserta cara penyelesaiannya, serta keterkaitan aplikasi dengan pilar smart city yang diangkat.
7. **Kesimpulan**
   Menuliskan ringkasan capaian praktikum dan pemahaman yang diperoleh, disusun dalam beberapa kalimat yang menjawab tujuan praktikum.
8. **Lampiran berisi Link Repositori GitHub**
   Mencantumkan tautan repositori GitHub yang dapat diakses oleh dosen pengampu, memuat kode program lengkap.
9. **Presentasi Hasil Praktikum**
   Mahasiswa mempresentasikan hasil praktikum di kelas, meliputi demonstrasi aplikasi yang berjalan dan penjelasan kode program, serta menjawab pertanyaan dari dosen pengampu.